CareerSphere AI: An Explainable, Skill-Aware Campus Placement Platform with Opportunity Math, Skill Roadmaps and a Grounded AI Copilot

A Report submitted in partial fulfillment of the requirements
for the Degree of

Bachelor of Technology

in

COMPUTER SCIENCE AND ENGINEERING

by

V. Venkata Rupin	**(2311CS010689)**
E. Manideep Reddy	**(2311CS010182)**
J. Pruthvi Sai	**(2311CS010264)**
S. Jaffer Ali	**(2311CS010639)**

Under the guidance of

[Guide name]

[Guide designation]



Department of Computer Science and Engineering

School of Engineering

MALLA REDDY UNIVERSITY

Maisammaguda, Dulapally, Hyderabad, Telangana 500100

2026

CareerSphere AI: An Explainable, Skill-Aware Campus Placement Platform with Opportunity Math, Skill Roadmaps and a Grounded AI Copilot

A Report submitted in partial fulfillment of the requirements
for the Degree of

Bachelor of Technology

in

COMPUTER SCIENCE AND ENGINEERING

by

V. Venkata Rupin	**(2311CS010689)**
E. Manideep Reddy	**(2311CS010182)**
J. Pruthvi Sai	**(2311CS010264)**
S. Jaffer Ali	**(2311CS010639)**

Under the guidance of

[Guide name]

[Guide designation]

Department of Computer Science and Engineering

School of Engineering

MALLA REDDY UNIVERSITY

Maisammaguda, Dulapally, Hyderabad, Telangana 500100

2026–2027

MALLA REDDY UNIVERSITY

(Telangana State Private Universities Act No.13 of 2020 and G.O.Ms.No.14, Higher Education (UE) Department)

Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Major Project report entitled CareerSphere AI: An Explainable, Skill-Aware Campus Placement Platform with Opportunity Math, Skill Roadmaps and a Grounded AI Copilot is submitted by V. Venkata Rupin (2311CS010689), E. Manideep Reddy (2311CS010182), J. Pruthvi Sai (2311CS010264) and S. Jaffer Ali (2311CS010639), to the Department of Computer Science and Engineering, Malla Reddy University, Hyderabad, in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering.

The report presents the problem, literature survey, objectives, methodology, testing, implementation, results and conclusions of the project 2026–2027.

INTERNAL GUIDE

Guide name

DEAN CSE

Dr. A.V.L.N. Sujith

External Examiner

MALLA REDDY UNIVERSITY

(Telangana State Private Universities Act No.13 of 2020 and G.O.Ms.No.14, Higher Education (UE) Department)

Department of Computer Science and Engineering

DECLARATION

We hereby declare that the project report entitled “CareerSphere AI: An Explainable, Skill-Aware Campus Placement Platform with Opportunity Math, Skill Roadmaps and a Grounded AI Copilot” has been prepared as part of our major project in the Department of Computer Science and Engineering, Malla Reddy University, Hyderabad, in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology.

Published literature, software resources and experimental records used in this work are acknowledged through citations and references. All data used in this report is fictional demo data created for the project. Prior submissions and individual contributions will be declared in accordance with the institution’s academic integrity requirements.

Place Hyderabad

Date DD-MM-2026

V. Venkata Rupin	**(2311CS010689)**
E. Manideep Reddy	**(2311CS010182)**
J. Pruthvi Sai	**(2311CS010264)**
S. Jaffer Ali	**(2311CS010639)**

ACKNOWLEDGEMENT

We express our sincere gratitude to Mr. Chamakura Malla Reddy, Chairman, and Dr V. S. K. Reddy, Vice Chancellor, Malla Reddy University, for the academic environment and institutional support provided for this project.

We thank our project guide, Guide name, Guide designation, Department of Computer Science and Engineering, for guidance, constructive feedback and support throughout the work.

We thank Dr. A.V.L.N. Sujith, Dean CSE, School of Engineering, and the faculty of the department for their encouragement. We thank the Training and Placement Cell of the university for explaining how campus placement works in practice, and we acknowledge the researchers and open-source communities whose work and software provide the foundations of this project.

We thank our families, friends and peers for their encouragement during the project.

V. Venkata Rupin	**(2311CS010689)**
E. Manideep Reddy	**(2311CS010182)**
J. Pruthvi Sai	**(2311CS010264)**
S. Jaffer Ali	**(2311CS010639)**

ABSTRACT

Campus placement portals record profiles, applications and results, but rarely tell anyone what to do next. This project designs, implements and evaluates CareerSphere AI, a placement platform for students, recruiters and the placement cell built on one transparent, level-aware match rule: the weighted share of a job's required skills a student has, where must-have skills count twice as much as nice-to-have ones and a skill below the level asked earns half credit, with a 70% "within reach" threshold and the recruiter's CGPA cutoff. Skill levels can be verified by server-graded assessments or by certificates checked by the placement cell. Four decision tools are built on the rule: Opportunity Math ranks each learning step by how many open jobs it would bring within reach; a Skill Roadmap searches every bundle of up to three steps exactly and marks steps that only pay off together; Hidden Talent shows recruiters consented students who fit a job but have not applied; and a Workshop Planner chooses k workshops for the batch by exact bit-mask search, because the objective is not submodular and greedy selection can fail. An AI Copilot with eight read-only tools and an evidence-checked résumé skill finder are grounded in the same data. On a fictional batch of 35 students and 12 jobs, Opportunity Math brought 1.43 jobs per student within reach against 1.03 for the most-demanded missing skill; the Skill Roadmap raised this to 4.71 with three steps and helped all 7 students for whom no single skill helps; and three exactly planned workshops gave all 35 students a job within reach where greedy selection reached 32, in 6.5 ms. The system is implemented with Spring Boot, React and MySQL and is covered by 39 unit tests and 17 API tests. These findings apply to the demo batch and the stated assumptions and do not establish an effect on real placements.

Keywords: campus placement; skill-gap analysis; explainable recommendation; combinatorial optimisation; large language models; Spring Boot; React.

TABLE OF CONTENTS

DESCRIPTION	PAGE NUMBER
ABSTRACT	VI
Chapter 1 Introduction	1
1.1 Outline of Project	1
1.2 Problem Definition	2
Chapter 2 Literature Survey	3
2.1 Survey	3
2.2 Previous Studies and Existing System	4
Chapter 3 Project Objectives	7
3.1 Objective of the Project	7
3.2 Limitations of the Project	9
Chapter 4 Design And Methodology	10
4.1 Proposed System	10
4.2 Architecture	12
4.3 Modules	13
4.4 Methods and Algorithms	14
4.5 Structural Design	17
4.6 System Design	19
4.7 Data Flow in the System	20
4.8 UML Diagram	21
Chapter 5 Testing	27
Need of Testing	27
Chapter 6 Code	29
6.1 System Implementation and outputs	32

TABLE OF CONTENTS CONTINUED

DESCRIPTION	PAGE NUMBER
Chapter 7 Results And Discussion	**44**
7.1 Results	**44**
7.2 Discussion	**45**
7.3 Existing system vs Proposed System	**46**
Chapter 8 Conclusion	**47**
8.1 Project Conclusion	**47**
Future Scope	**48**
References	**49**

LIST OF FIGURES

FIG NO	FIG NAME	PAGE NO
Figure 4.1	Proposed System	10
Figure 4.2	Architecture Diagram	12
Figure 4.3	Entity Relationship Diagram	17
Figure 4.4	Data Flow Diagram	20
Figure 4.5	Use Case Diagram	22
Figure 4.6	Class Diagram	23
Figure 4.7	Sequence Diagram	24
Figure 4.8	Activity Diagram	25
Figure 4.9	Deployment Diagram	26
Figure 6.1.1	Login Page	32
Figure 6.1.2	Career Map	33
Figure 6.1.3	Opportunity Math	33
Figure 6.1.4	Skill Roadmap	34
Figure 6.1.5	Job Detail with Explained Score	34
Figure 6.1.6	Placement Profile	35
Figure 6.1.7	Skill Assessment	35
Figure 6.1.8	Certifications	36
Figure 6.1.9	Career Copilot	36
Figure 6.1.10	Posting a Job	37
Figure 6.1.11	Ranked Applicants	38
Figure 6.1.12	Hidden Talent	38
Figure 6.1.13	Placement Analytics	39
Figure 6.1.14	Workshop Planner	39
Figure 6.1.15	Certificate Verification	40
Figure 6.1.16	Mobile View	40
Figure 6.1.17	Learning Recommendation Comparison	41
Figure 6.1.18	Workshop Planning Comparison	42
Figure 6.1.19	Runtime Comparison Graph	42
Figure 6.1.20	Hidden Talent per Job	43

LIST OF TABLES

TABLE NO	TABLE NAME	PAGE NO
Table 2.1	Literature perspectives and evaluation requirements	6
Table 4.1	Résumé score rules	16
Table 4.2	Hardware and software requirements	19
Table 5.1	Checks in the existing test suite	28
Table 7.1	Learning recommendations for 35 students	44
Table 7.2	Workshop plans: students reached / matches	44
Table 7.3	Hidden Talent across the 12 jobs	45
Table 7.4	Existing system and proposed system comparison	46

ABBREVIATIONS AND NOTATION

API Application Programming Interface

BCrypt Blowfish-based adaptive password hashing function

CGPA Cumulative Grade Point Average

DFD Data Flow Diagram

ER Entity Relationship

JPA Jakarta Persistence API

JSON JavaScript Object Notation

JWT JSON Web Token

LLM Large Language Model

REST Representational State Transfer

SQL Structured Query Language

UI User Interface

UML Unified Modeling Language

UUID Universally Unique Identifier

RQ Research Question

l_s(x) Level of student s in skill x (0 none, 1 Beginner, 2 Intermediate, 3 Advanced)

m_j(x) and w_j(x) Minimum level and weight (2 must-have, 1 nice-to-have) of skill x in job j

g_s and c_j CGPA of student s and CGPA cutoff of job j

k Number of workshops in a plan

B Bundle of learning actions in the Skill Roadmap

CHAPTER-1

INTRODUCTION

1.1 Outline of Project

Campus placement is the main route from an engineering college to a first job. Within a few months each year the training and placement cell invites companies, students apply to openings, recruiters shortlist and interview, and offers are made. A department may have several hundred final-year students and dozens of openings, each asking for its own mix of technical skills, a minimum CGPA and sometimes a particular branch [3].

Most colleges run this process on a placement portal where students keep a profile and résumé, companies post drives and the placement cell tracks applications. Such portals record what happened, but they rarely say what to do next. A student can see that a job asks for Java, Spring Boot and Git, yet cannot tell whether learning Spring Boot or Docker would open more doors across all current openings. A recruiter sees the applicants, but not the students who would fit and did not apply. The placement cell sees that many students lack a skill, but not which workshops would help the most students reach at least one job.

This project, **CareerSphere AI**, is a placement platform for students, recruiters and the placement cell built around one transparent rule: how much of a job's required skill set a student has, with must-have skills counting twice as much as nice-to-have ones and a skill below the level asked earning half credit [4]. Because the rule is exact and fast, it can be evaluated thousands of times a second to answer "what if" questions, and every answer can be explained line by line.

On top of the rule the platform offers Opportunity Math (the best next skill), a Skill Roadmap (the best two- and three-step learning plans), Hidden Talent (fitting students who did not apply), an exact Workshop Planner for the batch, verified skill levels, and an AI Copilot whose answers are grounded in the same data [8], [16]. This report establishes the problem, reviews related work, and documents the design, implementation, testing and measured findings on a fictional demo batch.

1.2 Problem Definition

The matching problem is to score every student against every open job so that the score can be checked by hand. A student has a level from 0 to 3 in each skill; a job lists required skills with a minimum level, marks each as must-have or nice-to-have, and may set a CGPA cutoff. A job is *within reach* when the score is at least 70% and the CGPA cutoff is met. A score that cannot be explained is not acceptable, because students, recruiters and the placement cell must be able to see why it is what it is.

The guidance problem is to turn the score into decisions. For a student, the question is which one skill, or which two or three skills together, would bring the most open jobs within reach for the least effort. Skills pay off in combination: a student who has only Git gains nothing from learning Java alone or Spring Boot alone for a job that needs all three (2 of 3 is 67%), but gains the job by learning both. Advice that looks at one skill at a time can therefore be wrong.

The planning problem is to choose k skill workshops for the whole batch so that the most students have at least one job within reach, and to invite each student only to the workshops they need. Because skills combine, this objective is not submodular and one-at-a-time greedy selection can miss the best plan [5], [7].

The trust problem is that skill claims are self-reported and AI output can state things nobody can check [10], [11]. Verified levels and grounded AI are needed so that a placement decision does not depend on an unverifiable statement. The central research problem is whether the exact, explainable approach gives better guidance than the usual heuristics while remaining fast enough to use interactively.

CHAPTER-2

LITERATURE SURVEY

2.1 Survey

The survey follows the research areas the project draws on: job recommendation and person–job fit, placement prediction, explainable recommendation, coverage problems and submodularity, tool use and grounding in language models, and fairness in algorithmic hiring. Its purpose is to position CareerSphere relative to established ideas and to identify what the evaluation must show.

This is a targeted review, not a systematic review with an exhaustive search protocol. The references provide context for mechanisms and constraints; the project does not reproduce every cited method or compare published numbers directly, because those studies use different data, objectives and populations.

The survey is organised by function rather than by a claim that one family replaces the others. Recommendation and fit models explain how matching is usually learned; explainable recommendation explains why an interpretable rule was chosen; coverage theory explains why workshop and learning-plan selection need exact search; and grounding studies explain how the AI features are constrained.

Existing placement portals, including the one used at our university, are also considered as an existing system. They provide profiles, résumé upload, placement consent, skill assessments, certificate verification, an introduction video and drive listings, which CareerSphere keeps and extends with decision support.

2.2 Previous Studies and Existing System

De Ruijt and Bhulai review a decade of job recommender systems and group methods into content-based, collaborative, hybrid and knowledge-based approaches [1]. They note that job recommendation is reciprocal (both the candidate and the employer must agree) and temporal (openings close), and that fairness has received limited attention. Zhu and co-authors learn joint neural representations of résumés and job postings to predict person–job fit [2]. Such models can be accurate when large labelled histories exist, but a college rarely has that data, and a learned score offers little explanation of an individual result.

Placement prediction studies, for example Alfana and co-authors [3], estimate the probability that a student is placed from academic and cognitive features. Prediction is useful for planning, but it describes a situation rather than changing it: a 40% chance of placement does not say which skill to learn.

Zhang and Chen distinguish post-hoc explanations, produced after a black-box prediction, from models that are interpretable by construction [4]. CareerSphere is of the second kind: the matched skills, missing skills, level gaps and CGPA rule are the computation itself.

2.2 Previous Studies and Existing System continued

Choosing k items to cover as many users as possible is a maximum coverage problem. For monotone submodular objectives the greedy algorithm achieves a 1 − 1/e approximation [5], which is the best possible in polynomial time unless P = NP [6]; Krause and Golovin survey the area [7]. The 70% threshold in this project creates complementarities between skills, so the objective is not submodular and the greedy guarantee does not apply.

Toolformer [8] and ReAct [9] show that language models can decide when to call external tools and use the results; commercial APIs expose the same pattern as function calling [16]. Ji and co-authors survey hallucination [10], and Gao and co-authors show that even strong models often fail to support their statements with correct citations [11]. CareerSphere therefore restricts the Copilot to read-only tools over the student's own data and verifies résumé evidence by exact string matching.

Raghavan and co-authors examine the claims of vendors of algorithmic pre-employment assessments and the practical limits of removing bias [12]. CareerSphere uses no demographic attributes; its inputs are skills, levels and the recruiter's published CGPA cutoff, and every score can be audited by hand. Security follows standard practice: signed JSON Web Tokens [13], BCrypt password hashing [14] and the OWASP guidance [20].

2.2 Previous Studies and Existing System comparison

The selected references address different parts of the problem. Some motivate the matching rule, some explain the planning objective, and some constrain the AI features. A comparison matrix is useful when it preserves these differences rather than reducing every study to a single ranking.

Table 2.1 Literature perspectives and evaluation requirements

Research area	References	Implication
Job recommendation and fit	[1], [2]	Learned scores need history and are hard to explain
Placement prediction	[3]	Predicts outcomes; does not choose actions
Explainable recommendation	[4]	Use a rule that is its own explanation
Coverage and submodularity	[5], [6], [7]	Thresholds break greedy; search exactly
Tool use and grounding	[8]–[11], [16]	Answer only from tools; check evidence
Fairness and security	[12], [13], [14], [20]	No demographic inputs; standard auth

The matrix identifies what the evaluation must show. Guidance claims need a comparison with the usual heuristic of recommending the most-demanded missing skill. Planning claims need a comparison with greedy selection and simple rankings, plus runtime. AI claims need the grounding mechanism to be visible. The existing portal is the operational baseline: CareerSphere keeps its workflow and adds the decision layer.

CHAPTER-3

PROJECT OBJECTIVES

3.1 Objective of the Project

The first objective is to build a working platform for three roles (student, recruiter and placement cell) with secure login, profiles, résumé upload, job posting with skill levels, applications and status tracking, so that every later feature runs on real records rather than on a mock-up.

The second objective is to define an explainable, level-aware match score with must-have and nice-to-have skills and a clear “within reach” rule that includes the recruiter’s CGPA cutoff. The same rule must be used by every feature, so that a student, a recruiter and the placement cell always see consistent numbers.

The third objective is to turn the score into decisions: Opportunity Math and a Skill Roadmap for students, ranked applicants and Hidden Talent for recruiters, and an exact Workshop Planner for the placement cell. Skill combinations must be searched exactly, because one-at-a-time advice can miss plans whose skills only pay off together.

The fourth objective is to make skill levels trustworthy and the AI grounded. Levels can be verified by server-graded assessments and by certificates checked by the placement cell; the Copilot may only read data through tools, and the résumé finder may only suggest skills whose evidence is in the résumé.

The final objective is to test and evaluate the system: automated unit and API tests, a browser walk-through of every screen, and a comparison of each method with simple baselines on a demo batch, and to make the project easy to run from Terminal, VS Code, Xcode, Android Studio and Docker.

3.1.1 Research Questions

The study is organised around four questions that connect the proposed methods to evidence that can be inspected. Each question has a corresponding comparison in the results.

RQ1: How many more open jobs does the action chosen by Opportunity Math bring within reach of a student, compared with recommending the missing skill that most jobs ask for?

RQ2: How much does an exact search over plans of two and three learning steps add to the best single step, and how often do those plans depend on skills that only pay off together?

RQ3: Does exact search over workshop combinations reach more students than greedy, one-at-a-time selection and simple demand-based rankings, and is it fast enough to use interactively?

RQ4: How many consented students fit a job, or are one step away from it, without having applied, and can recruiters reach them without weakening the consent rule?

The questions permit favourable, unfavourable and inconclusive answers. A method that does not beat its baseline on some measure remains informative, and the results report every comparison, including where a simple ranking matches the exact plan.

3.2 Limitations of the Project

All results come from a fictional demo batch of 35 students, 2 companies, 12 jobs and 22 skills. They show how the methods behave; they do not measure the effect on real placements, which would need a study over a real placement season. Real student records are not used, in line with the AI provider's free-tier terms.

The weights (2 for must-have, 1 for nice-to-have), the half credit and the 70% threshold are design choices, not learned from data. Unverified levels are self-reported, and the bank of 120 assessment questions supports screening rather than certification.

The Workshop Planner assumes that a workshop teaches its skill to every invitee and ignores capacity and cost. The Skill Roadmap counts effort in levels gained, not in hours. Exact search is practical at college scale; much larger batches would need an integer program or the greedy fallback that the planner already provides.

Interview scheduling with calendars, offer letters, notifications by email and communication with companies outside the platform are outside the scope of this project. The human evaluation kit is prepared but the labelling is not yet complete.

CHAPTER-4

DESIGN AND METHODOLOGY

4.1 Proposed System

The proposed system separates the inputs, one shared match rule, and the decisions built on that rule. Profiles, skill levels, résumés, certificates and jobs enter the system; assessments and certificate reviews turn self-reported levels into verified ones; the rule scores every student against every open job; and decisions are computed from those scores for students, recruiters, the placement cell and the AI assistant.

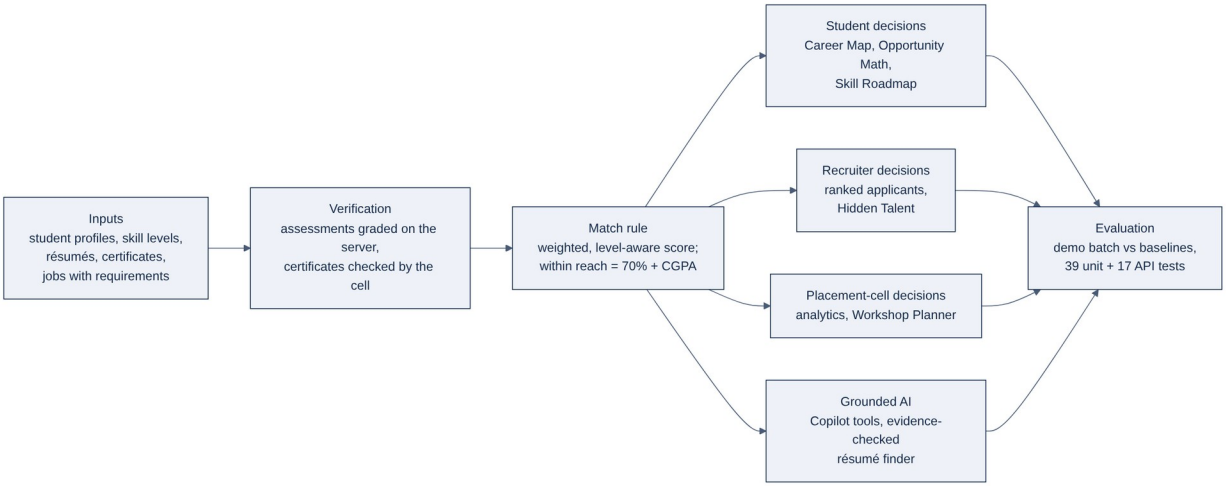


Fig 4.1 Proposed System

Every decision uses the same rule, so a student, a recruiter and the placement cell always see consistent numbers. Learning plans and workshop plans are searched exactly over skill combinations instead of being chosen one skill at a time, and every number is shown with the sentence that explains it.

The system keeps the familiar workflow of the university placement portal (résumé, consent, assessment, introduction video, applications) and adds the decision layer on top. It does not predict placement outcomes and does not use any demographic attribute.

4.1.1 Problem Formulation

Let a student s have level $\ell_s(x) \in \{0, 1, 2, 3\}$ in each skill x (0 means not known; 1 to 3 are Beginner, Intermediate and Advanced) and CGPA g_s. Let an open job j require the skill set R_j, with minimum level $m_j(x)$, weight $w_j(x) = 2$ for a must-have and 1 for a nice-to-have skill, and an optional CGPA cutoff c_j. The credit of one skill and the match score are

$$\kappa(\ell, m) = 1 \text{ if } \ell \geq m; \quad \tfrac{1}{2} \text{ if } 0 < \ell < m; \quad 0 \text{ if } \ell = 0 \tag{4.1}$$

$$\text{score}(s, j) = \text{round}\left(100 \cdot \Sigma_x \, w_j(x) \cdot \kappa(\ell_s(x), m_j(x)) \,/\, \Sigma_x \, w_j(x) \right) \tag{4.2}$$

with both sums over the required skills $x \in R_j$. A job is within reach when

$$\text{reach}(s, j) = [\, \text{score}(s, j) \geq 70 \,] \wedge [\, g_s \geq c_j \,] \tag{4.3}$$

In code the sums are kept in half units and the percentage is (200 · half + total) div (2 · total), which rounds halves up without floating point. With default jobs (all must-have, Beginner) the threshold admits 3 of 4 and 4 of 5 required skills but not 2 of 3. *Example:* for Java at Intermediate (must), Spring Boot (must) and Git (nice), a student with Java at Beginner, Spring Boot and Git earns 2 + 4 + 2 = 8 of 10 half units, so the score is 80% and Java is reported as a level gap.

4.2 Architecture

The architecture separates the clients, security, the REST controllers, the services, the rule engine, the AI layer and storage. Every request passes Spring Security, which checks the JSON Web Token and the role before a controller is reached.

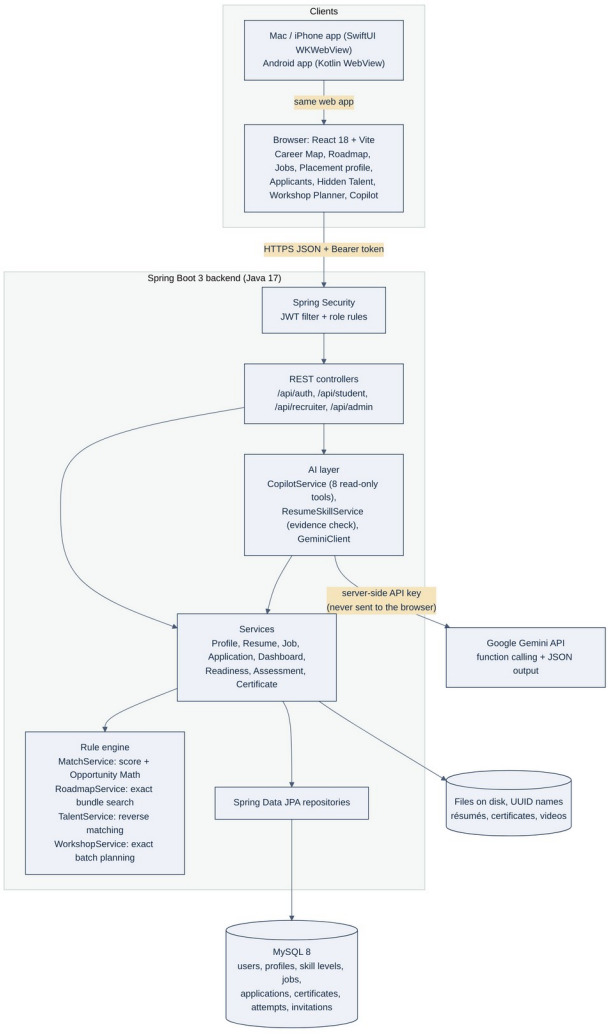


Fig 4.2 Architecture Diagram

Controllers call services, which hold the business rules; services read and write MySQL through Spring Data JPA and keep uploaded files on disk under random names. The rule engine computes every score and plan. The AI layer never writes data on its own: the Copilot reads data only through tools that call the same services, and the Gemini key stays on the server.

4.3 Modules

The **authentication module** (AuthService, JwtService, SecurityConfig) registers students and recruiters, issues signed tokens valid for eight hours, and restricts each URL prefix to one role. Admin accounts cannot be created through the public register endpoint.

The **student profile module** (StudentProfileService, ResumeService, ReadinessService) stores branch, CGPA, marks, links, headline and skills with levels, handles résumé upload with type and signature checks, placement consent and the introduction video, and computes the résumé score and readiness points. The **verification module** (AssessmentService, QuestionBank, CertificateService) runs timed assessments graded on the server and routes certificates to the placement cell for review.

The **jobs and applications module** (JobService, ApplicationService) lets recruiters post jobs with minimum levels and must-have or nice-to-have flags, lets students apply once per job, and ranks applicants with explanations. The **guidance module** (MatchService, DashboardService, RoadmapService) provides the Career Map, Opportunity Math, the what-if simulator, the Skill Roadmap and the next best action. The **Hidden Talent module** (TalentService) finds consented students who fit a job without having applied and records invitations.

The **placement-cell module** (WorkshopService, AdminController) provides analytics, the demand–supply skill gap, the batch leaderboard, certificate review and the Workshop Planner. The **AI module** (CopilotService, CopilotTools, ResumeSkillService, GeminiClient) provides the Copilot and the résumé skill finder, with an offline mode when no key is set. The React frontend has one page per screen and a small service layer for API calls.

4.4 Methods and Algorithms

Opportunity Math. An action is learning a skill x to a level t, or raising a known skill to a level t that some open job asks for. With L_s the student's levels and reach(L, j) equal to (4.3) evaluated with levels L, the gain of an action is

$$\text{gain}_s(x, t) = |\ \{\ j : \neg\text{reach}(L_s, j) \wedge \text{reach}(L_s \oplus (x, t), j)\ \}\ | \tag{4.4}$$

Actions are ranked by gain, ties broken by the number of open jobs requiring the skill. The what-if simulator applies the same rule to any set of actions, and on the Career Map each job is drawn at distance 100 − score from the student.

Skill Roadmap. The roadmap searches every bundle B of 1, 2 and 3 actions, at most one per skill, and keeps for each size the bundle with the most jobs within reach and then the least effort

$$\text{effort}(B) = \Sigma_{(x,\,t)\,\in\,B}\ (\ t - \ell_s(x)\ ) \tag{4.5}$$

A bundle is shown only if it beats every smaller one; it is marked *works together* when its gain exceeds the sum of the single gains (4.4) of its steps. For each nearby job the roadmap also finds the cheapest subset of that job's gaps, or names the CGPA cutoff when no skill can help. With about 30 actions a student needs roughly 10^3 bundle checks.

Algorithm 4.1: Skill Roadmap for one student

```
input: levels L, CGPA g, open jobs J
A ← every (skill, target level) some job asks for above L
for k ← 1 to 3 do
    best ← bundle of k actions, one per skill: most jobs within reach, then least effort
    if reach(best) > reach of every smaller bundle shown then keep it; flag works together
for each job j not within reach: CGPA blocker, or cheapest subset of its gaps
return bundles, paths to the closest jobs, number of bundles checked
```

4.4.1 Workshop Planning and Hidden Talent

Let S be the students, J the open jobs and C the candidate skills. A plan $T \subseteq C$ has k workshops; a workshop raises every attendee to at least Intermediate. With $n_s(T)$ the number of jobs within reach of s after the plan, the objectives are

$$f_{reach}(T) = |\{ s : n_s(T) > 0 \}|, \quad f_{match}(T) = \Sigma_s\, n_s(T) \tag{4.6}$$

Reach every student maximises (f_{reach}, f_{match}) in that order; *Most matches overall* maximises (f_{match}, f_{reach}). The objective is not submodular: for one student with {Git} and one job requiring {Java, Spring Boot, Git}, f({Java}) = f({Spring Boot}) = 0 while f({Java, Spring Boot}) = 1, so the greedy guarantee [5] does not hold.

The planner therefore enumerates all C(|C|, k) subsets. Students and jobs are encoded as bit masks (one per level and weight class), so each test of (4.3) costs a few OR, AND and population-count operations. If C(|C|, k) · |S| · |J| would exceed 3×10^8 it falls back to greedy and says so. Each student is invited only to the smallest subset of the plan that gives them the same result.

Hidden Talent runs the rule from the job's side. For a recruiter's job it scores every student who gave placement consent and has not applied, lists those within reach, and lists those for whom one action would bring the job within reach, with that action named. An invitation appears on the student's Jobs page; a unique (job, student) constraint prevents repeats.

4.4.2 Verification and Grounded AI

An assessment draws 8 questions (4 basic, 4 advanced) for one of 15 skills from a bank of 120, shuffles the options and stores their order on the server; the browser never receives the answers. Eight correct answers verify Advanced, six or seven Intermediate and four or five Beginner; below four nothing is verified and a retake waits 24 hours. A certificate verified by the placement cell adds its skill at Intermediate. The résumé score is computed from ten fixed rules (Table 4.1), and readiness points add 15 per verified skill, 10 per verified certificate and 5 per job within reach.

Table 4.1 Résumé score rules

Rule	Points	Maximum
Résumé uploaded	15	15
About you: 150+ characters (5 points for 50+)	10	10
Skills listed, up to 5	4 each	20
Skills at Intermediate or above, up to 5	2 each	10
Skills verified, up to 3	5 each	15
Verified certificates, up to 2	5 each	10
GitHub link / LinkedIn link	5 / 5	10
Introduction video; CGPA, 10th and 12th marks	5 / 5	10

The Copilot uses Gemini function calling [16] with eight read-only tools; the server runs each tool for the student identified by the token, proposed actions run only when the student clicks them, and every answer lists the data used. The résumé finder keeps a suggestion only if its quoted line occurs in the résumé text [11]; without a key, word-boundary patterns are used (JavaScript is not Java).

4.5 Structural Design

The structural design separates accounts, profiles, the skill catalogue, jobs and the records that link them. The `users` table holds every account with a role (STUDENT, RECRUITER or ADMIN). Students and recruiters each have a one-to-one profile. Skills are a shared catalogue.

A student's skills are stored with a self-reported level and, separately, the verified level, which locks the level against editing. A job's skills are stored with a minimum level and a must-have or nice-to-have flag. Applications and invitations each link one job and one student with a unique constraint on the pair, so a student cannot apply twice and a recruiter cannot invite the same student twice, even under simultaneous requests.

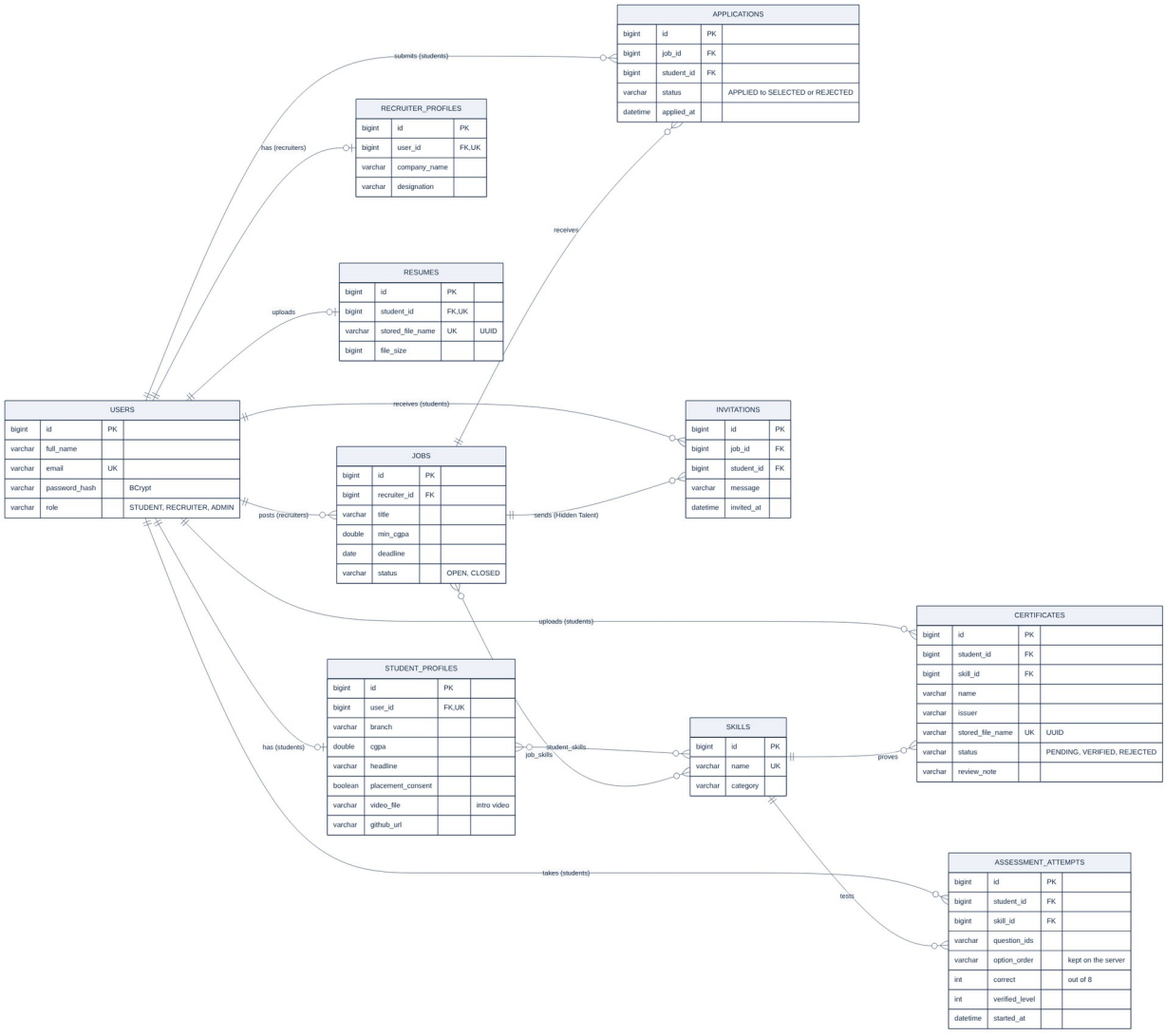


Fig 4.3 Entity Relationship Diagram

Certificates (with issuer, status and review note) and assessment attempts (with question identifiers, option order, correct answers and the verified level) belong to a student and a skill. Hibernate creates the 16 tables from the entity classes; uploaded résumés, certificates and videos are stored under `backend/uploads/` with random UUID names.

4.6 System Design

The system accepts requests from the React web app, the Mac and iPhone app and the Android app. Every request carries a JSON Web Token; each URL prefix (`/api/student`, `/api/recruiter`, `/api/admin`) is open to one role only, and services also check ownership, so a recruiter sees only applicants to their own jobs.

Outputs are JSON: scores with explanations, Career Map positions, Opportunity Math rankings, roadmap bundles, ranked applicants, Hidden Talent lists, analytics and workshop plans. Errors return a status code with a plain sentence and, for forms, a message per field. Uploads are checked by size and by the file's first bytes. The environment used for development and testing is listed in Table 4.2.

Table 4.2 Hardware and software requirements

Component	Minimum	Used in development
Processor / memory	Dual-core, 4 GB RAM	Apple M-series, 8 GB RAM or more
Operating system	macOS, Windows or Linux	macOS 14 or later
Java / build	JDK 17, Maven 3.9	OpenJDK 17, Maven (Homebrew)
Node.js	Node 18, npm	Node 20 LTS
Database	MySQL 8	MySQL 8.4 (Homebrew)
Editors	Any	VS Code, Xcode 15+, Android Studio
AI (optional)	Gemini API key	Gemini free tier, demo data only

The project is technically, operationally and economically feasible: every component is free and widely taught, the exact searches run in milliseconds on a laptop, the workflow follows the existing portal, and online hosting costs a few dollars a month. Without an AI key the Copilot and résumé finder run in offline mode, so the system never depends on an external service.

4.7 Data Flow in the System

Data enter as profiles, résumés, assessment answers, certificates, jobs and applications. Authentication attaches the user and role; the profile and verification process stores levels; the matching and planning process scores every student against every job and produces plans; and the results return to each user.

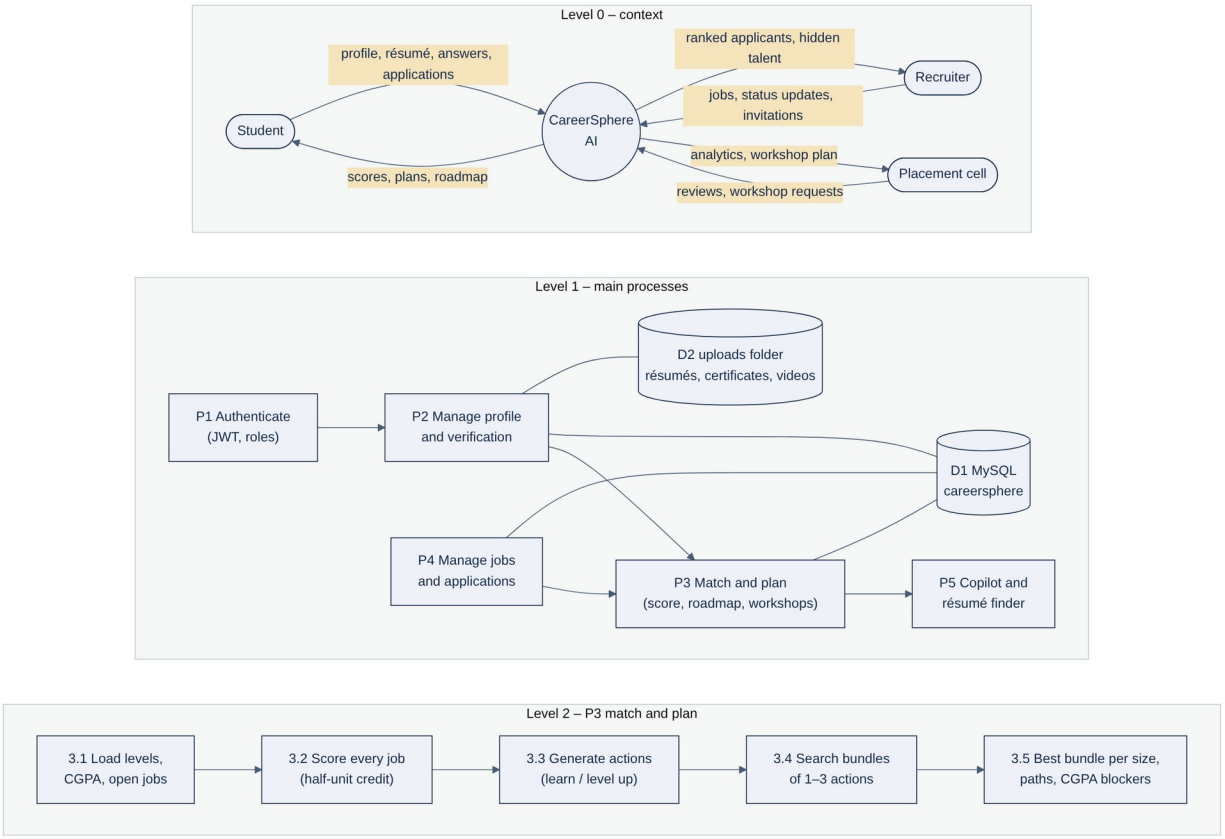


Fig 4.4 Data Flow Diagram

At Level 0 the system exchanges data with students, recruiters and the placement cell. At Level 1 there are five processes (authenticate, manage profile and verification, match and plan, manage jobs and applications, Copilot and résumé finder) and two stores (the MySQL database and the uploads folder). At Level 2 matching and planning loads levels and jobs, scores every job in half units, generates actions, searches bundles of one to three actions, and returns the best bundle of each size.

4.8 UML Diagram

The UML diagrams document the structure and behaviour of the implementation: who uses the system (use case), how the code is organised (class), how a request travels (sequence), how a student moves through the placement flow (activity), and where each part runs (deployment).

4.8.1 Use Case Diagram

The actors are the student, the recruiter and the placement cell, with the Gemini service as an external actor used by the Copilot and the résumé finder. Use cases shown in green are the new decision-support features.

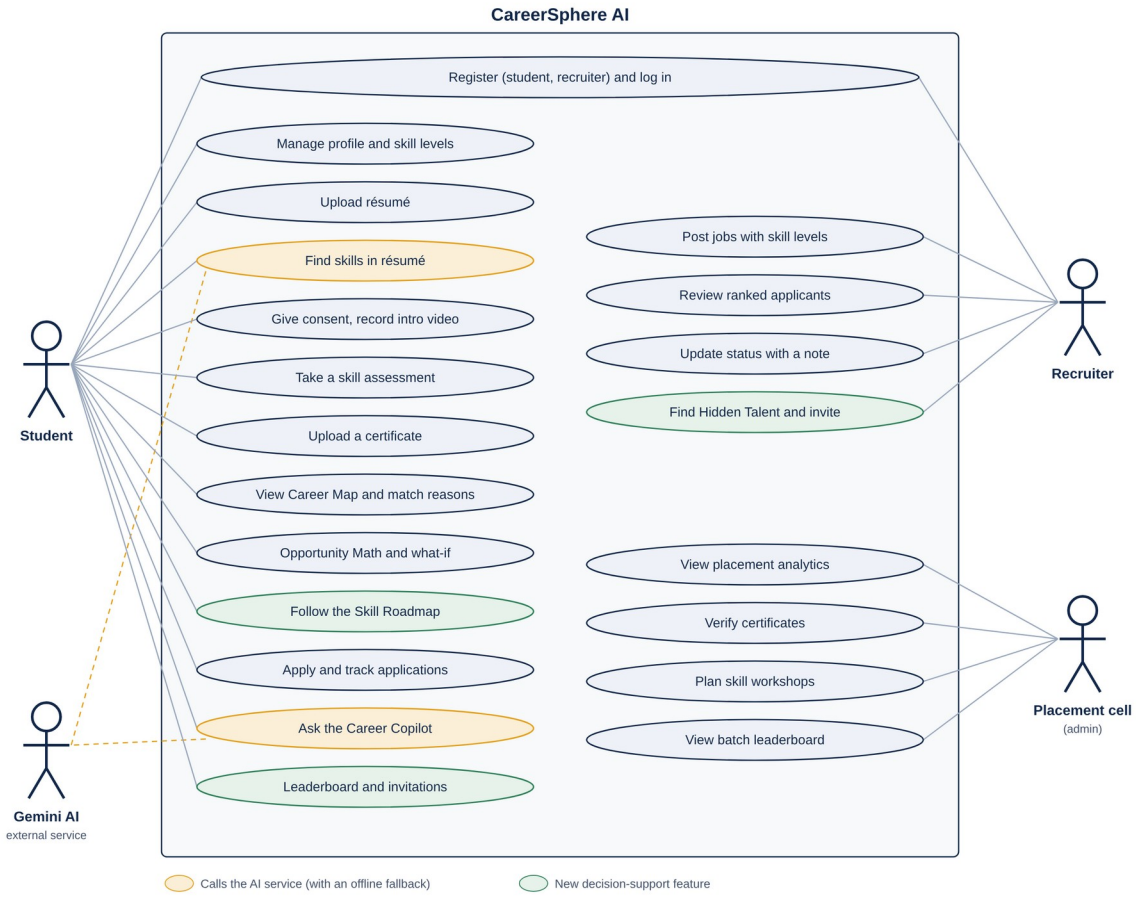


Fig 4.5 Use Case Diagram

Students manage their placement profile and use the guidance features; recruiters post jobs and review applicants and hidden talent; the placement cell verifies certificates, views analytics and plans workshops.

4.8.2 Class Diagram

The class diagram shows the principal entities and the services of the rule engine. Multiplicity labels identify logical associations; secondary helpers and data-transfer records are omitted to keep it readable.

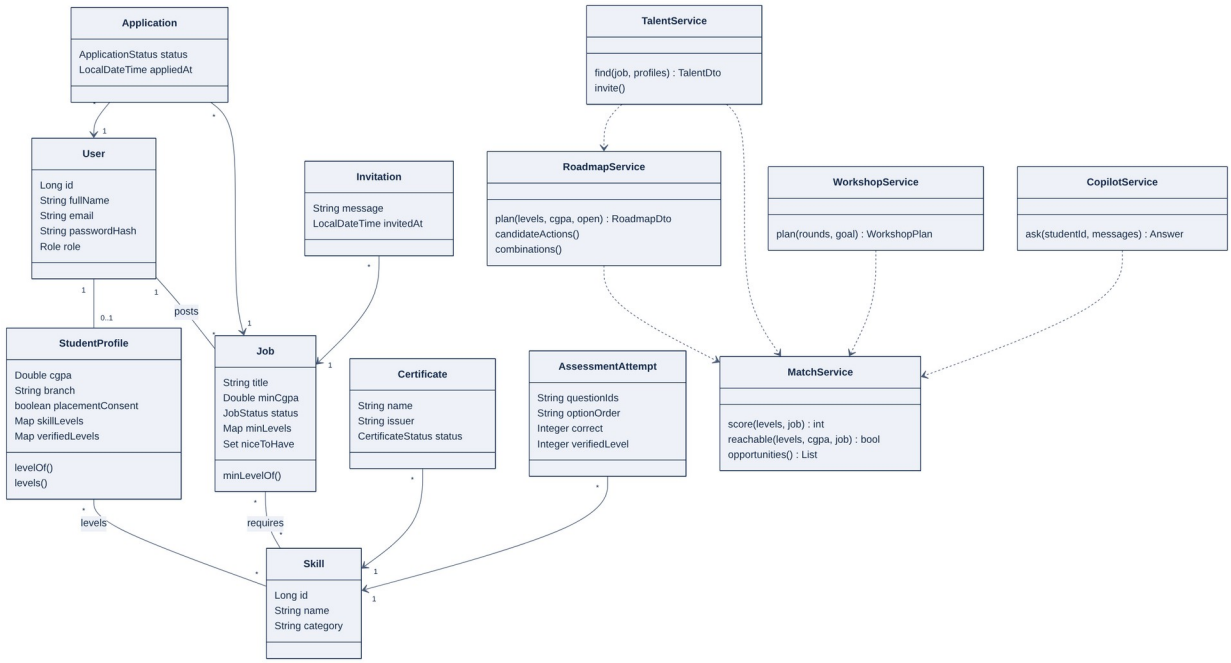


Fig 4.6 Class Diagram

StudentProfile and Job are linked to Skill with levels; Application, Invitation, Certificate and AssessmentAttempt link students to jobs and skills. RoadmapService, TalentService, WorkshopService and CopilotService all depend on MatchService, so every feature uses the same rule.

4.8.3 Sequence Diagram

The sequence diagram summarises a normal session: the student logs in, opens the Skill Roadmap, applies to a job within reach, and the recruiter reviews ranked applicants and updates the status.

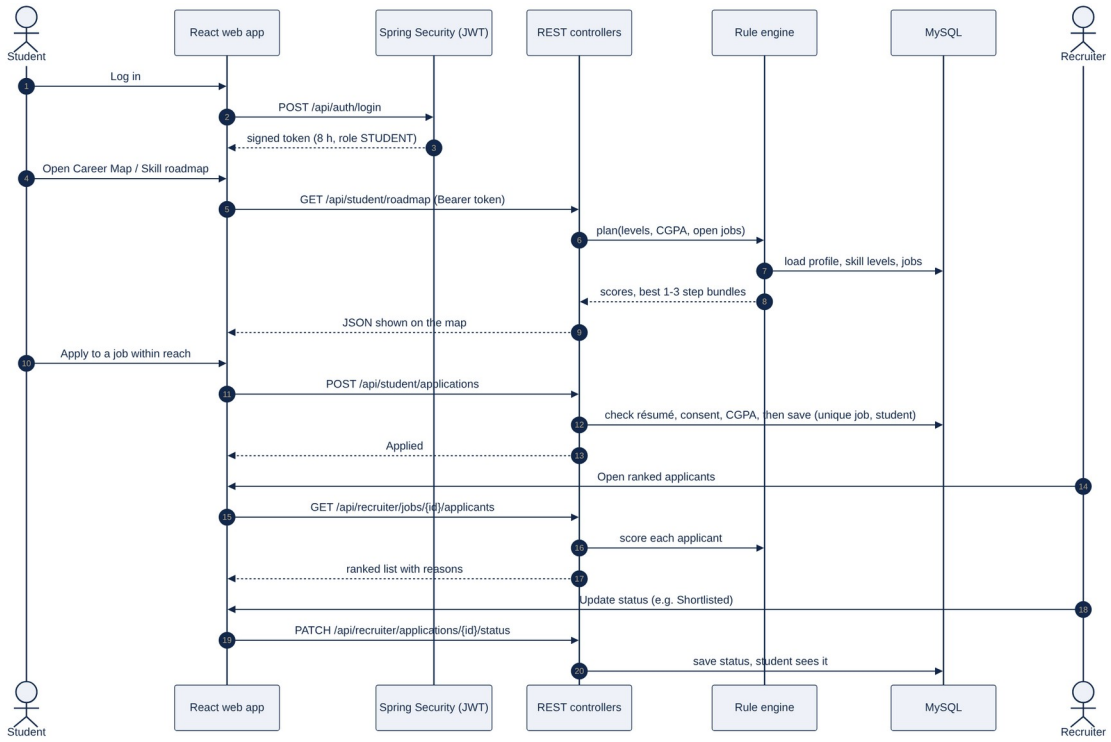


Fig 4.7 Sequence Diagram

Every call after login carries the token; the rule engine computes scores and plans from the database; and the unique (job, student) constraint is the final guard against duplicate applications.

4.8.4 Activity Diagram

The activity diagram shows the student's path from login through résumé, consent and verification to applying, with the loop back through the Skill Roadmap or a workshop when no job is within reach yet.

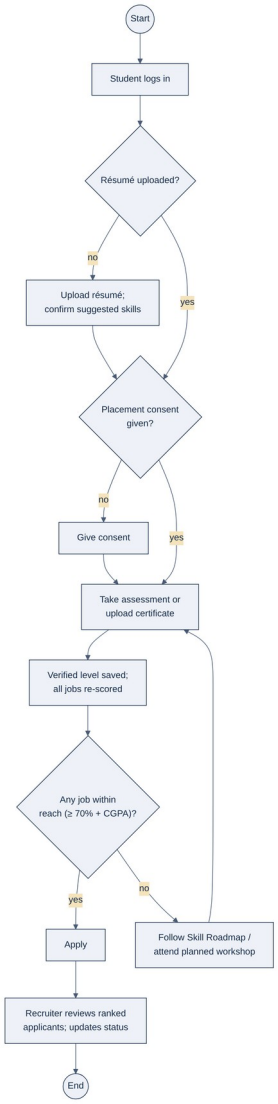


Fig 4.8 Activity Diagram

A verified level immediately re-scores every job, so the decision "any job within reach?" always uses current data.

4.8.5 Deployment Diagram

The deployment diagram separates the development set-up on a Mac, the Android emulator, the external Gemini service and the online deployment.

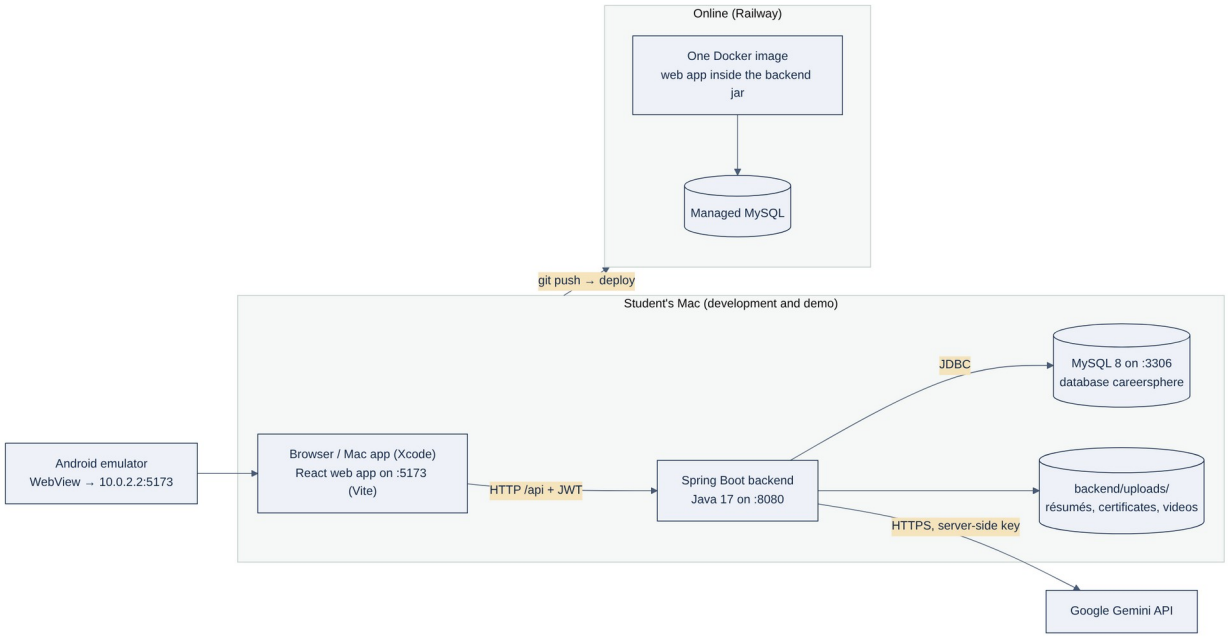


Fig 4.9 Deployment Diagram

Locally, the browser or Mac app reaches the React app on port 5173, which calls Spring Boot on port 8080; MySQL runs on port 3306 and files sit in `backend/uploads/`. Online, one Docker image carries the web app inside the backend jar, with a managed MySQL database. The Gemini API is called from the server only.

CHAPTER-5

TESTING

Need of Testing

Testing is necessary because every number CareerSphere shows is used to make a decision: which skill to learn, whom to shortlist, which workshop to run. A wrong rule would mislead students and the placement cell without any visible error. A single apparent improvement can be invalidated by a rounding mistake, a CGPA rule applied to the score instead of separately, or a threshold that is not inclusive.

The test suite therefore has three levels. **Unit tests** check the rules in isolation on small, hand-checkable cases, so every expected value can be worked out on paper. **API tests** start the whole application on an in-memory H2 database and call the real endpoints through the real security filter chain with MockMvc. **User-interface checks** walk through every screen in a real browser at desktop and phone widths and fail on any script error.

Security tests check that each role can reach only its own APIs, that a recruiter cannot update applications to another recruiter's job, that admin accounts cannot be self-registered, and that uploads are checked by their first bytes rather than their file name. Assessment tests check that the browser never receives the answers and that grading happens on the server.

These checks improve confidence in specific behaviours, but passing them is not a proof that every path is correct or that the methods are superior. The comparisons with baselines in Chapter 7 supply the evidence for the methods themselves.

Testing and verification evidence

The unit-test suite was executed during preparation of this report: all 39 unit tests passed. As a check that the tests can detect a faulty rule, the threshold comparison was changed from ≥ to >; the inclusive-threshold test failed at once and passed again when the change was reverted. The 17 end-to-end API tests (TC-01 to TC-17) run with `mvn test` against a fresh in-memory database.

Table 5.1 Checks in the existing test suite

Area	Tests	Property checked
Match rule and Opportunity Math	9 + 6	Share of skills, rounding, CGPA rule, inclusive 70% threshold, half credit, nice-to-have weight, best action first
Skill Roadmap	3	Two-step bundle when no single skill helps; least effort wins; cheapest path and CGPA blocker
Hidden Talent	1	Only consented students who have not applied
Workshop Planner	6	Three workshops reach all students; greedy fails on a complementary case
Placement profile	8	Résumé score ≤ 100; step order; answers never sent; 24-hour retake
Résumé finder	6	Evidence lines; JavaScript is not Java; ordinary words not mistaken
API (TC-01 to TC-17)	17	Login, roles, uploads, applying, consent, assessments, certificates, roadmap, invitations

The browser walk-through logs in as each role, takes and submits an assessment, sends a Hidden Talent invitation and checks that it appears on the student's Jobs page, plans workshops, and repeats the walk at a phone width of 390 pixels, confirming that no page scrolls sideways. Invalid input returns status 400 with a message per field, and unexpected errors return a generic message so that no stack trace reaches the browser.

CHAPTER-6

CODE

Source Code match score

The following is an exact excerpt from `MatchService` in `backend/src/main/java/com/careersphere/service/MatchService.java`. It computes the weighted, level-aware score in integer half units and the within-reach rule used by every feature.

```
public static int weight(Job job, Skill skill) {
    return job.isNiceToHave(skill) ? 1 : 2;
}
/** Credit in half units: 2w if level >= needed, w if 0 < level < needed, 0 if m
issing. */
public static int halfCredit(int level, int needed, int weight) {
    if (level <= 0) return 0;
    return level >= needed ? 2 * weight : weight;
}
/** round(100 x half / total), half up, in integers. */
public static int percent(long half, long total) {
    if (total <= 0) return 100;
    return (int) ((200L * half + total) / (2L * total));
}
public int score(Map<Long, Integer> levels, Job job) {
    long half = 0, total = 0;
    for (Skill s : job.getSkills()) {
        int w = weight(job, s);
        total += 2L * w;
        half += halfCredit(levels.getOrDefault(s.getId(), 0), job.minLevelOf(s),
w);
    }
    return percent(half, total);
}
public boolean reachable(Map<Long, Integer> levels, Double cgpa, Job job) {
    return cgpaEligible(cgpa, job) && score(levels, job) >= MATCH_THRESHOLD;
}
```

Integer arithmetic keeps the planner, the roadmap and the explanation shown to the user in exact agreement. The CGPA check is separate from the score, so a student can see that a job is a good skill match but closed to them.

Source Code skill roadmap

The following excerpt from `RoadmapService.plan` walks every combination of k learning actions, at most one per skill, and keeps the best bundle of each size.

```
for (int k = 1; k <= Math.min(MAX_STEPS, distinctSkills(actions)); k++) {
    List<Action> best = null;
    int bestReach = -1, bestEffort = Integer.MAX_VALUE;
    for (List<Action> combo : combinations(actions, k)) {
        checked++;
        int r = reach(apply(levels, combo), cgpa, open);
        int effort = combo.stream().mapToInt(Action::effort).sum();
        if (r > bestReach || (r == bestReach && effort < bestEffort)) {
            best = combo; bestReach = r; bestEffort = effort;
        }
    }
    // A size can gain nothing while the next size gains a lot: keep looking.
    if (best == null || bestReach <= lastReach) continue;
    bundles.add(describe(levels, cgpa, open, before, now, best, single));
    lastReach = bestReach;
}
```

The `continue` rather than `break` matters: when no single skill helps, the two-step bundle that does help must still be found. The `describe` step orders the steps and marks a bundle as working together when it beats the sum of its single gains.

Source Code hidden talent and workshop search

The first excerpt is simplified from `TalentService.find`. It skips students without consent and students who already applied, then lists students within reach and those one action away.

```
for (StudentProfile p : all) {
    if (!p.isPlacementConsent()) continue;
    if (applications.existsByJobIdAndStudentId(job.getId(), p.getUser().getId())
) continue;
    Map<Long, Integer> levels = p.levels();
    int score = match.score(levels, job);
    if (!match.cgpaEligible(p.getCgpa(), job)) continue;
    if (score >= MatchService.MATCH_THRESHOLD) { reach.add(candidate(p, score, n
ull)); continue; }
    for (RoadmapService.Action a : RoadmapService.candidateActions(levels, List.
of(job)))
        if (match.reachable(RoadmapService.apply(levels, List.of(a)), p.getCgpa(
), job)) {
            near.add(candidate(p, score, a.label())); break;
        }
}
```

The second excerpt is the recursion of the exact Workshop Planner in `WorkshopService`. Each chosen skill sets one bit; at full depth every student is tested against every job with bit masks.

```
for (int i = from; i <= b.skills() - (size - depth); i++) {
    current[depth] = i;
    System.arraycopy(masks[depth], 0, masks[depth + 1], 0, b.words);
    masks[depth + 1][i >>> 6] |= 1L << (i & 63);
    search(b, goal, depth + 1, i + 1, size, current, masks, best, bestScore);
}
```

Assessment grading is a fixed rule in `AssessmentService.levelFor`: 8 correct answers give level 3, 6 or 7 give level 2, 4 or 5 give level 1, and fewer give 0 with a 24-hour retake wait. The complete classes remain in the repository.

6.1 System Implementation and outputs

The implementation runs with `./run.sh` from the project folder, or from VS Code with Cmd + Shift + B, which starts MySQL, the Spring Boot backend on port 8080 and the React web app on port 5173. The data live in the MySQL database `careersphere`; uploaded files are kept in `backend/uploads/`. The following figures are taken from the running application with the demo data.

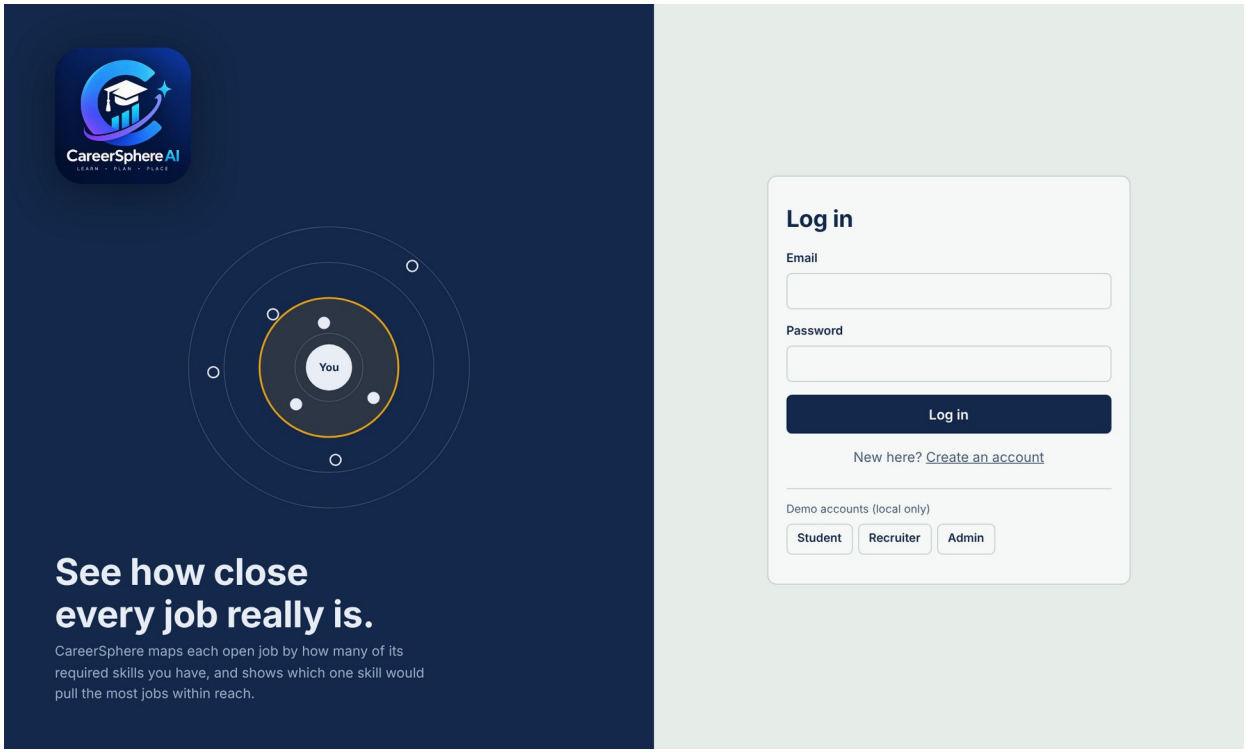


Fig 6.1.1 Login Page

Student Outputs

The student sees every open job on the Career Map, the best next skill, the Skill Roadmap and the placement profile.

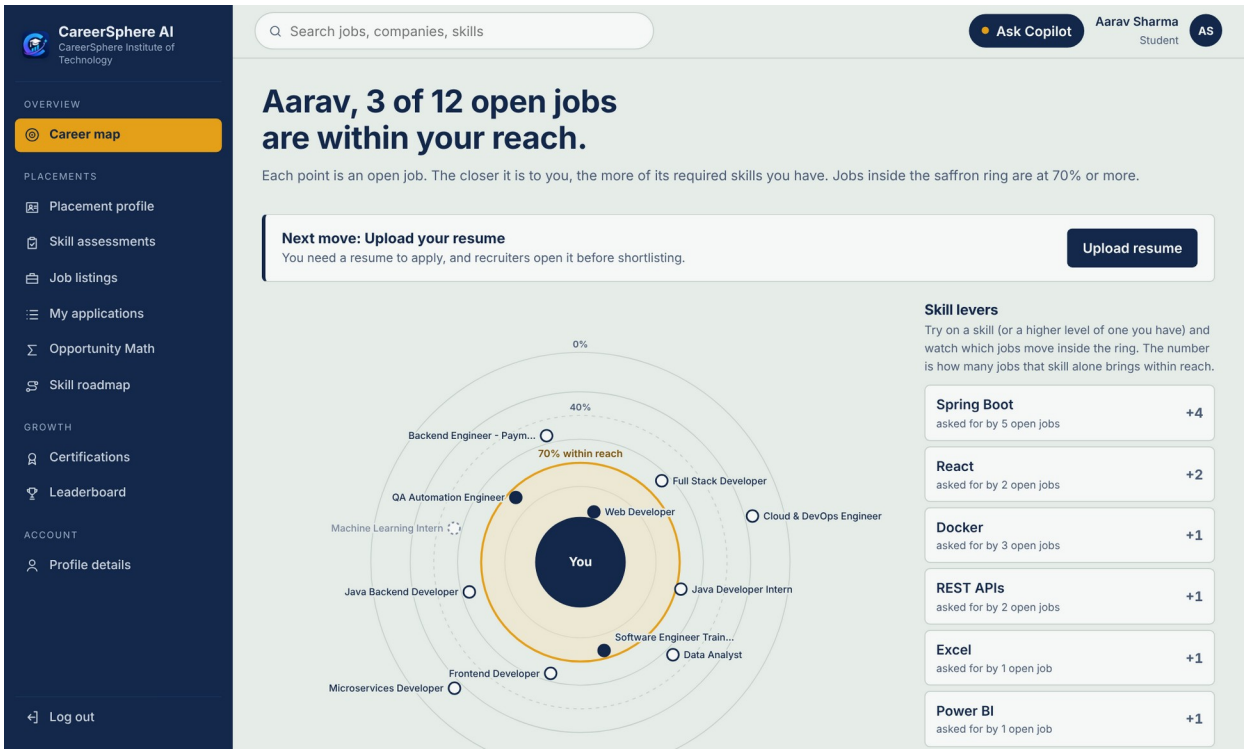


Fig 6.1.2 Career Map

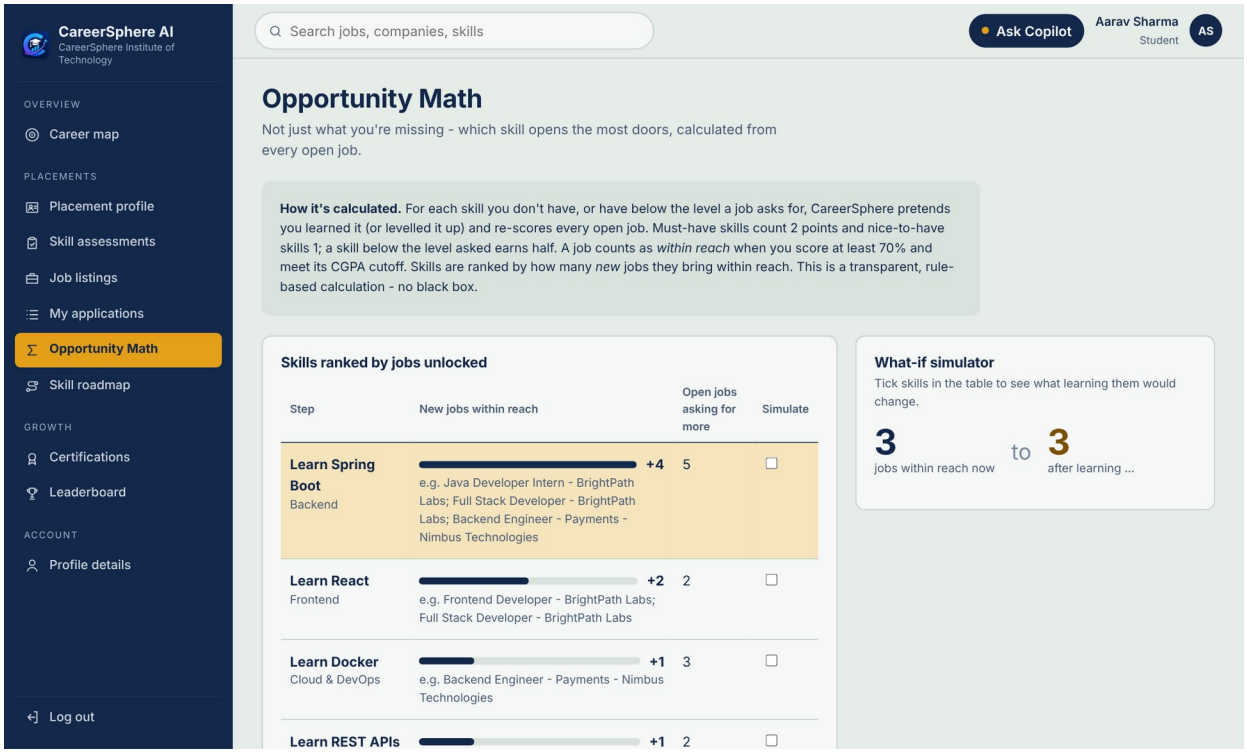


Fig 6.1.3 Opportunity Math

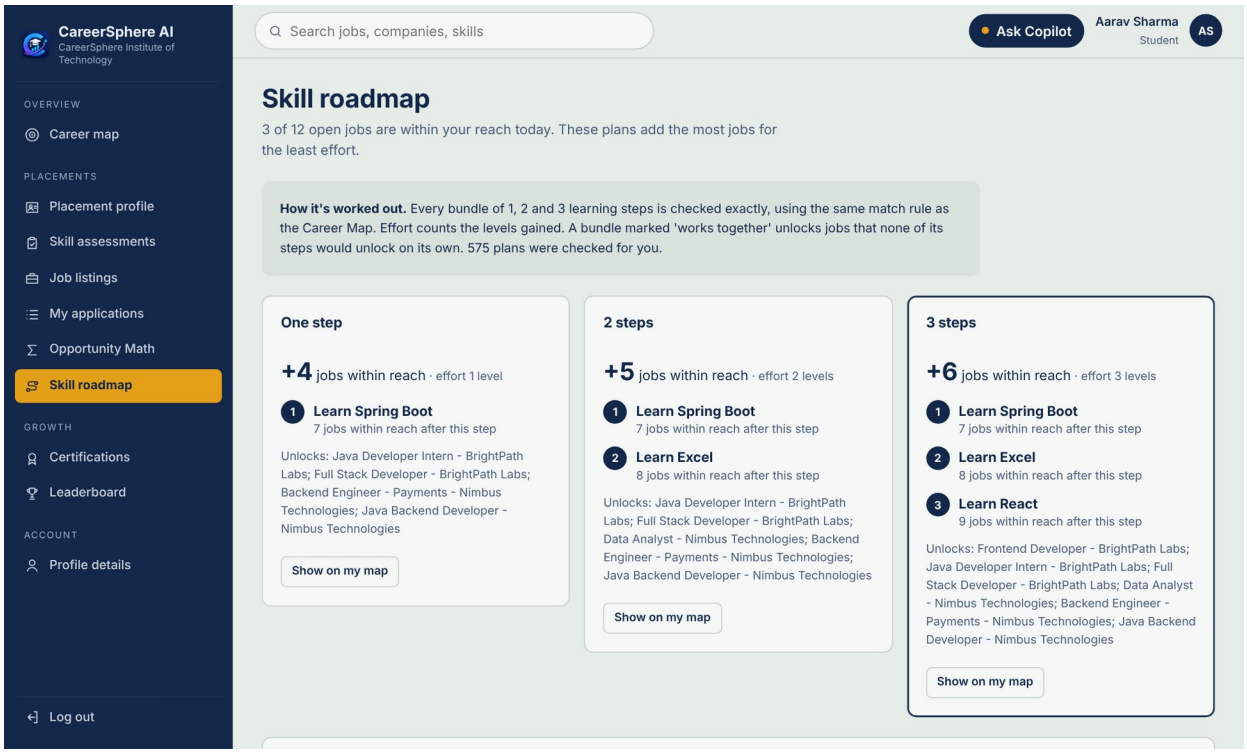


Fig 6.1.4 Skill Roadmap

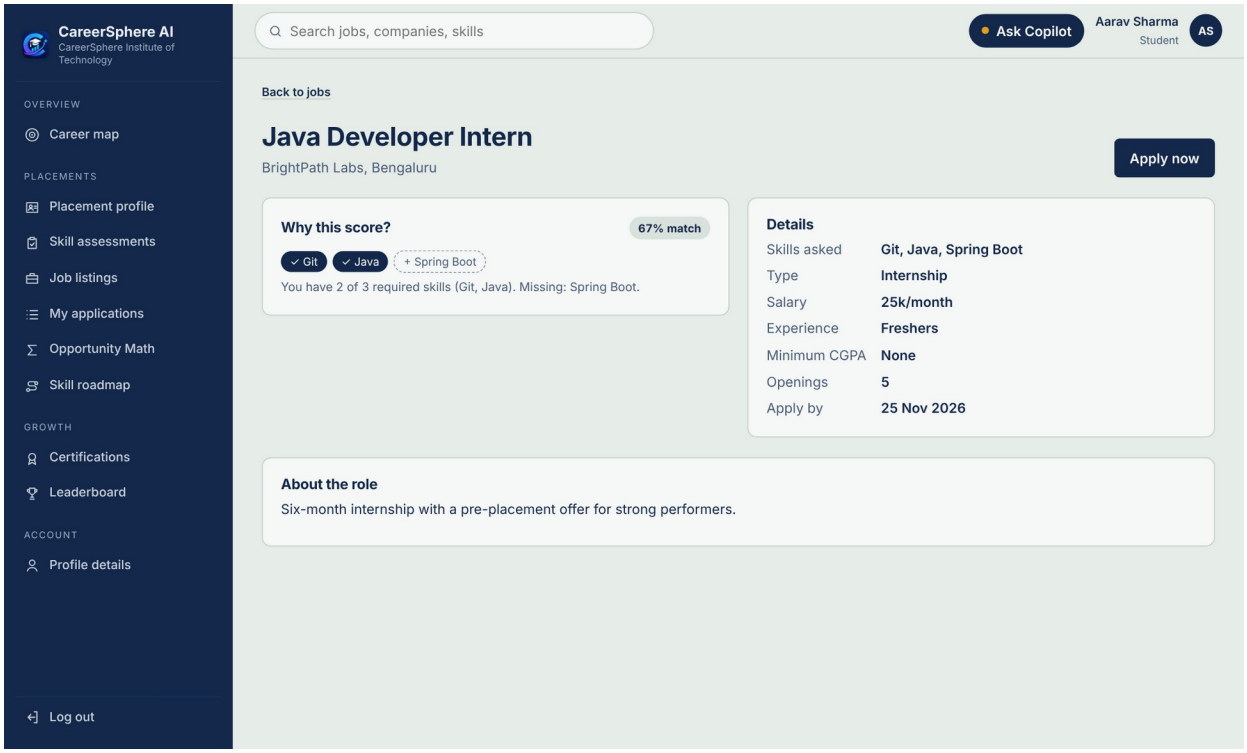


Fig 6.1.5 Job Detail with Explained Score

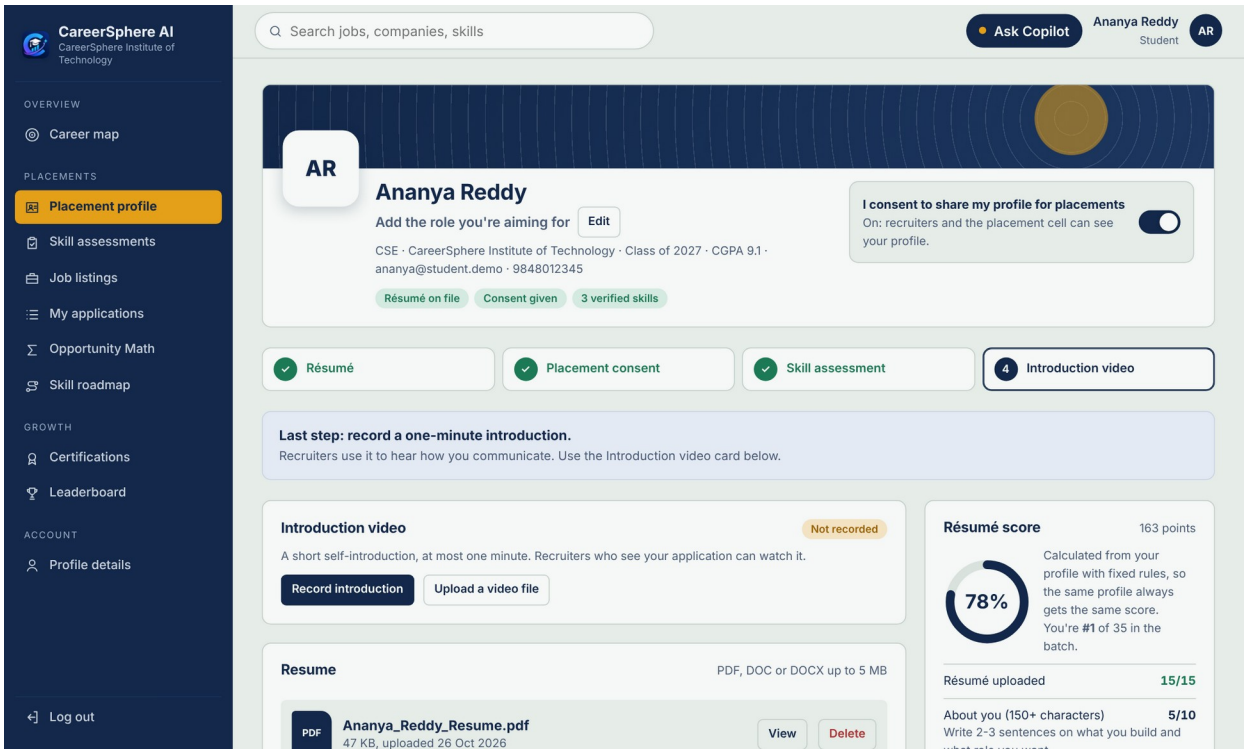


Fig 6.1.6 Placement Profile

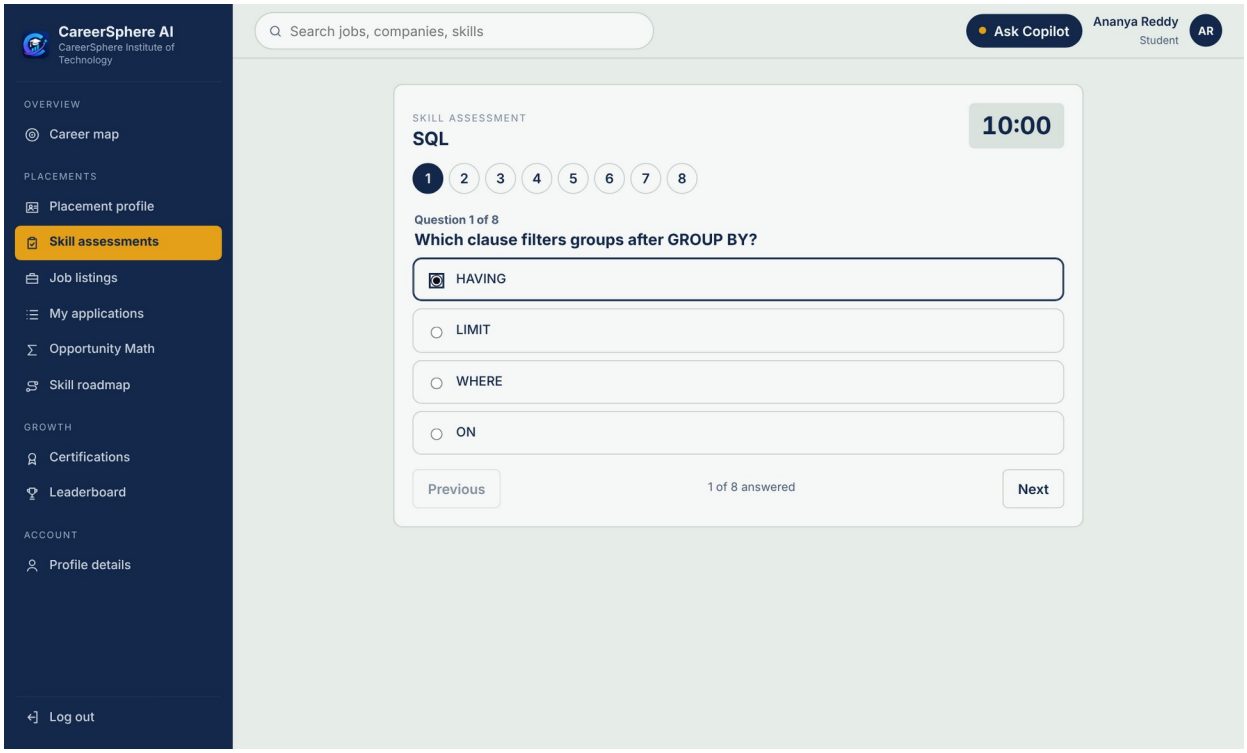


Fig 6.1.7 Skill Assessment

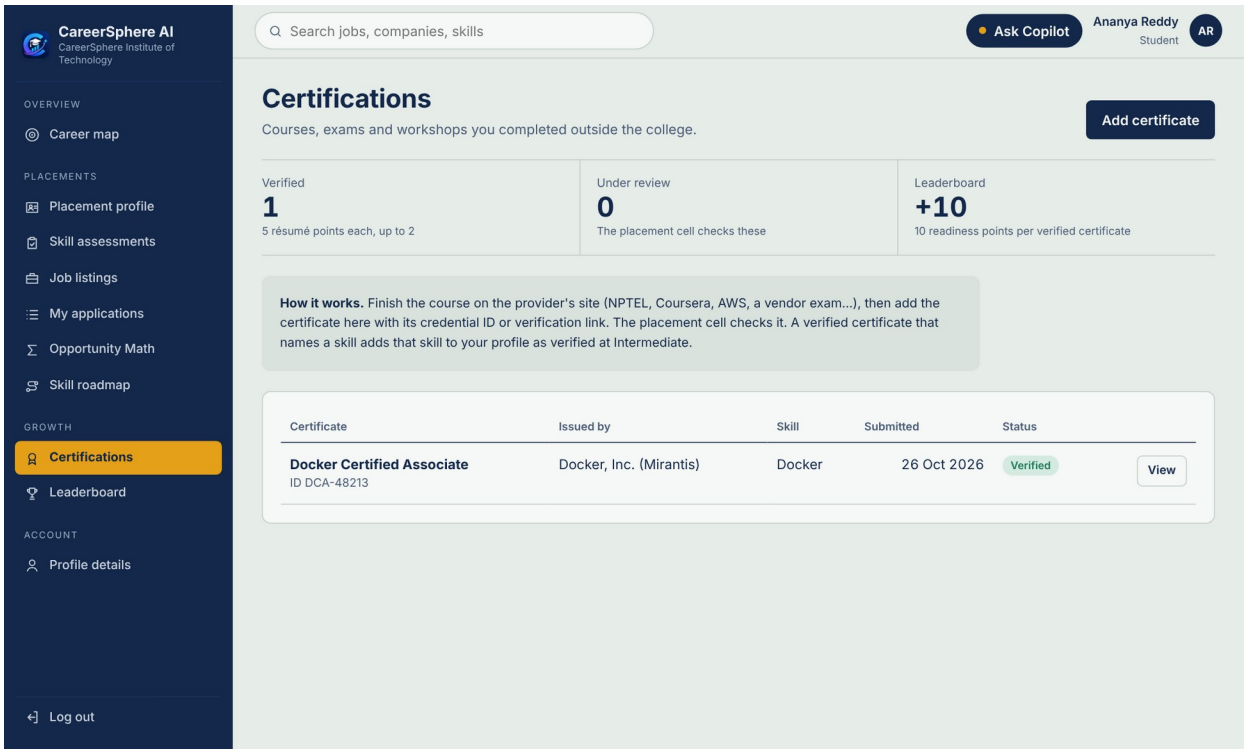


Fig 6.1.8 Certifications

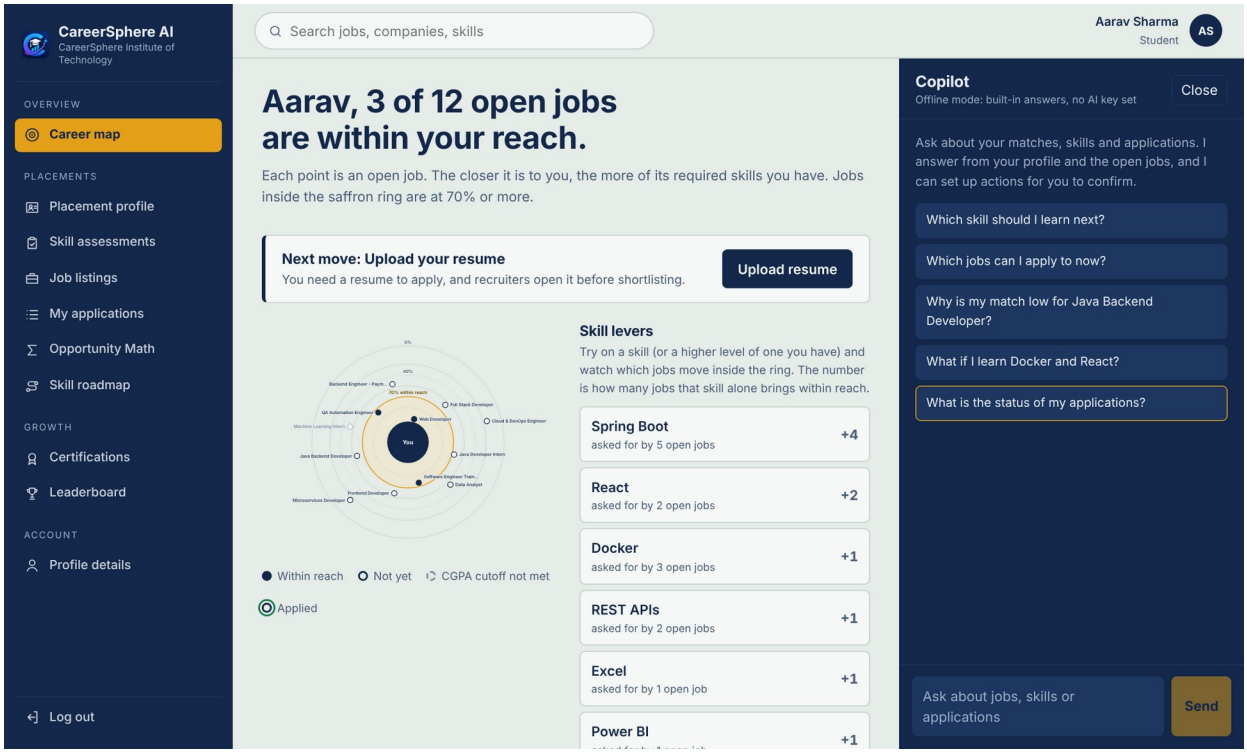


Fig 6.1.9 Career Copilot

Recruiter and Placement Cell Outputs

The recruiter posts jobs with skill levels, reviews ranked applicants and invites hidden talent; the placement cell verifies certificates, views analytics and plans workshops.

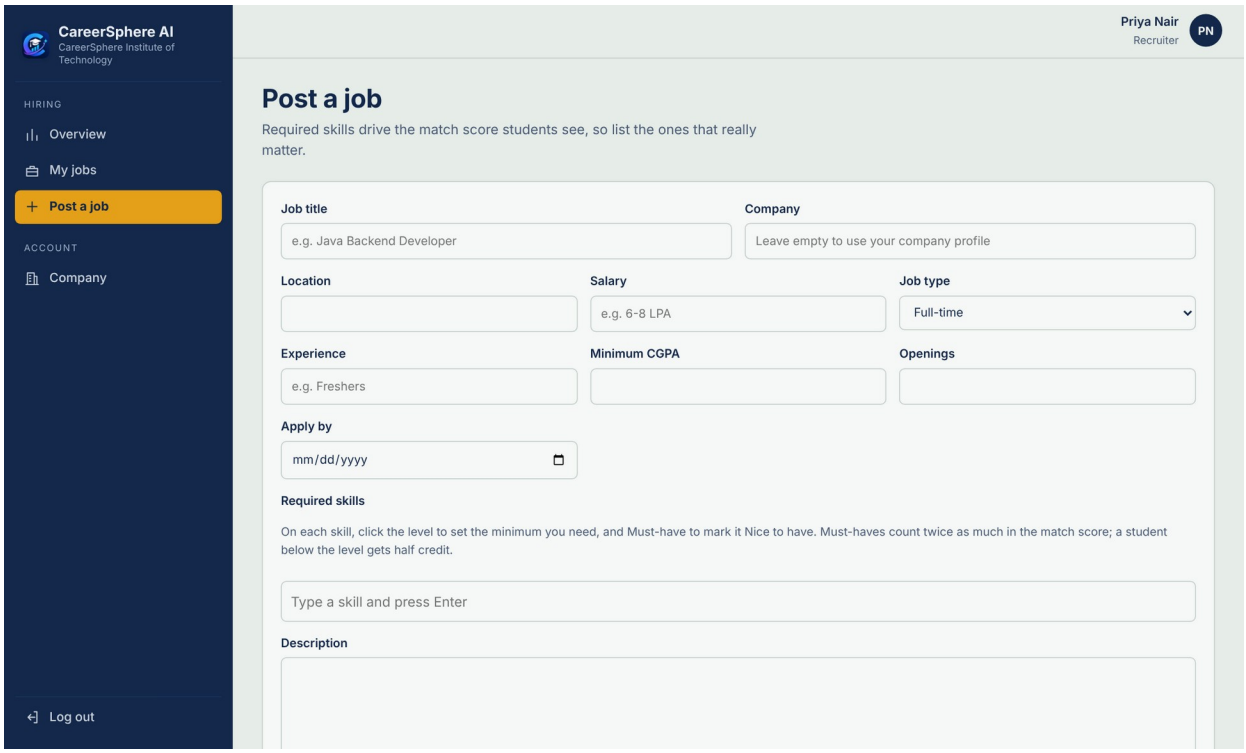


Fig 6.1.10 Posting a Job

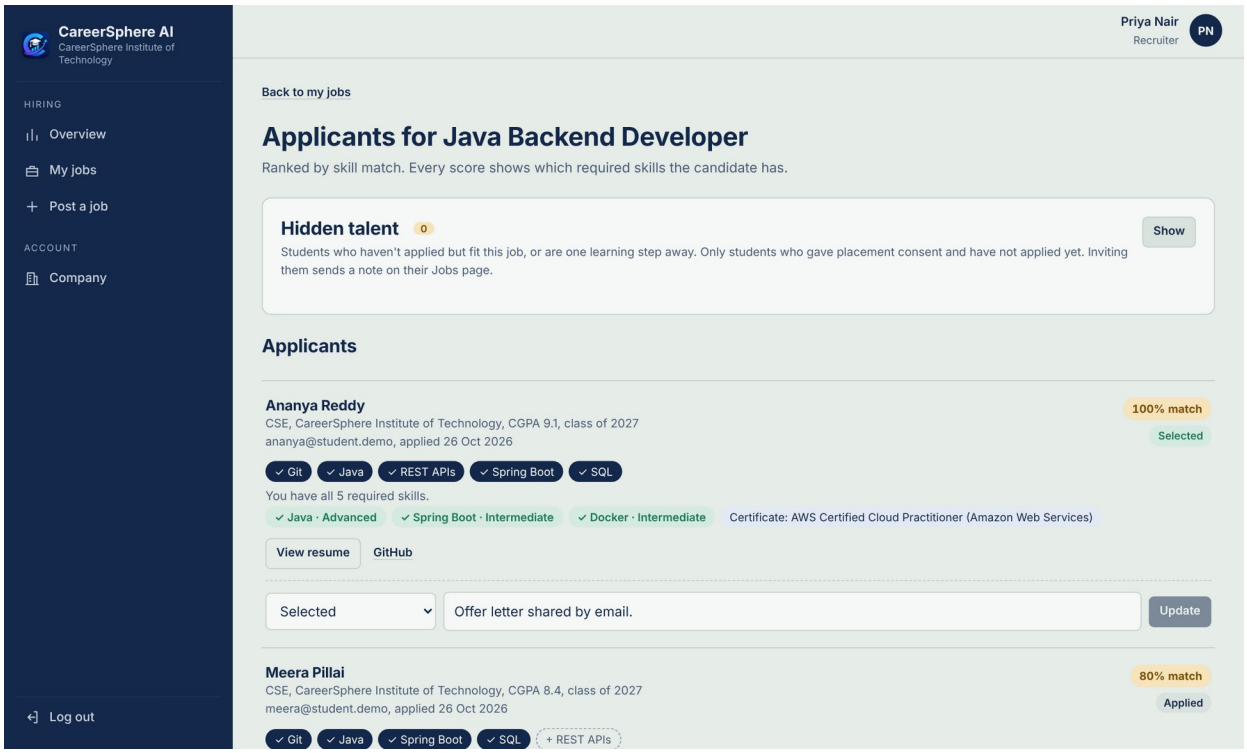


Fig 6.1.11 Ranked Applicants

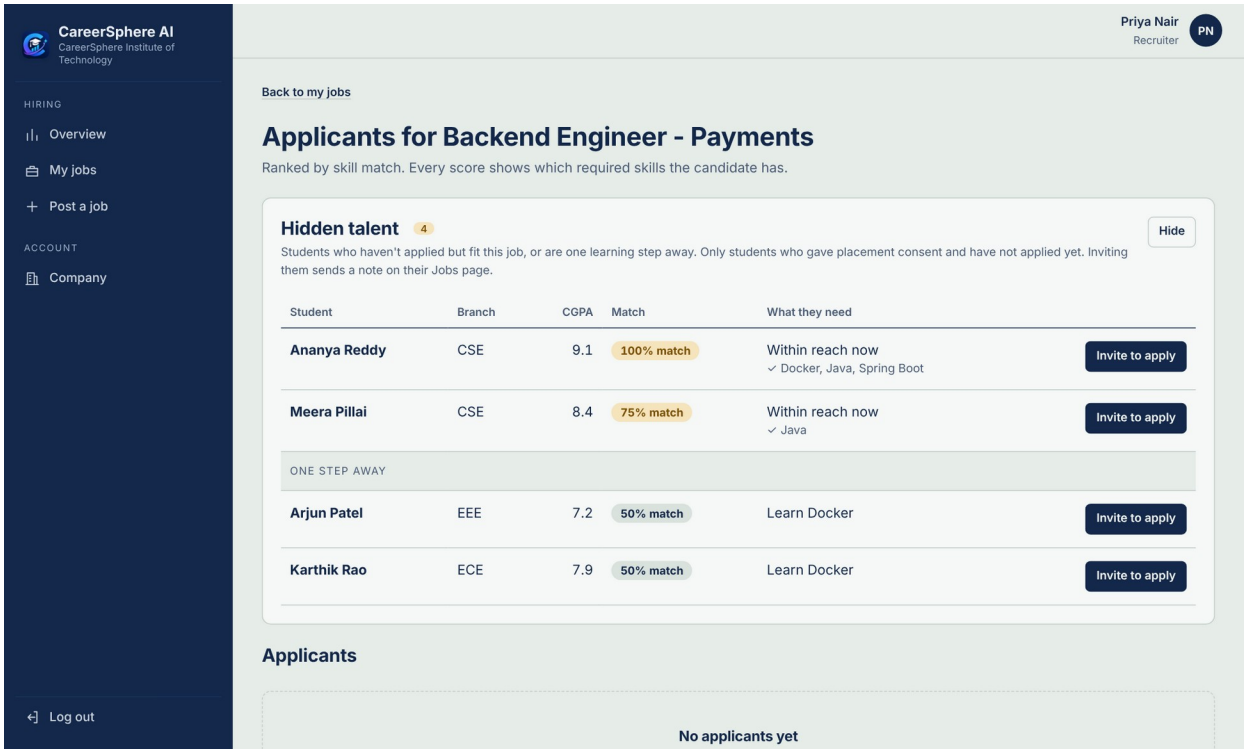


Fig 6.1.12 Hidden Talent

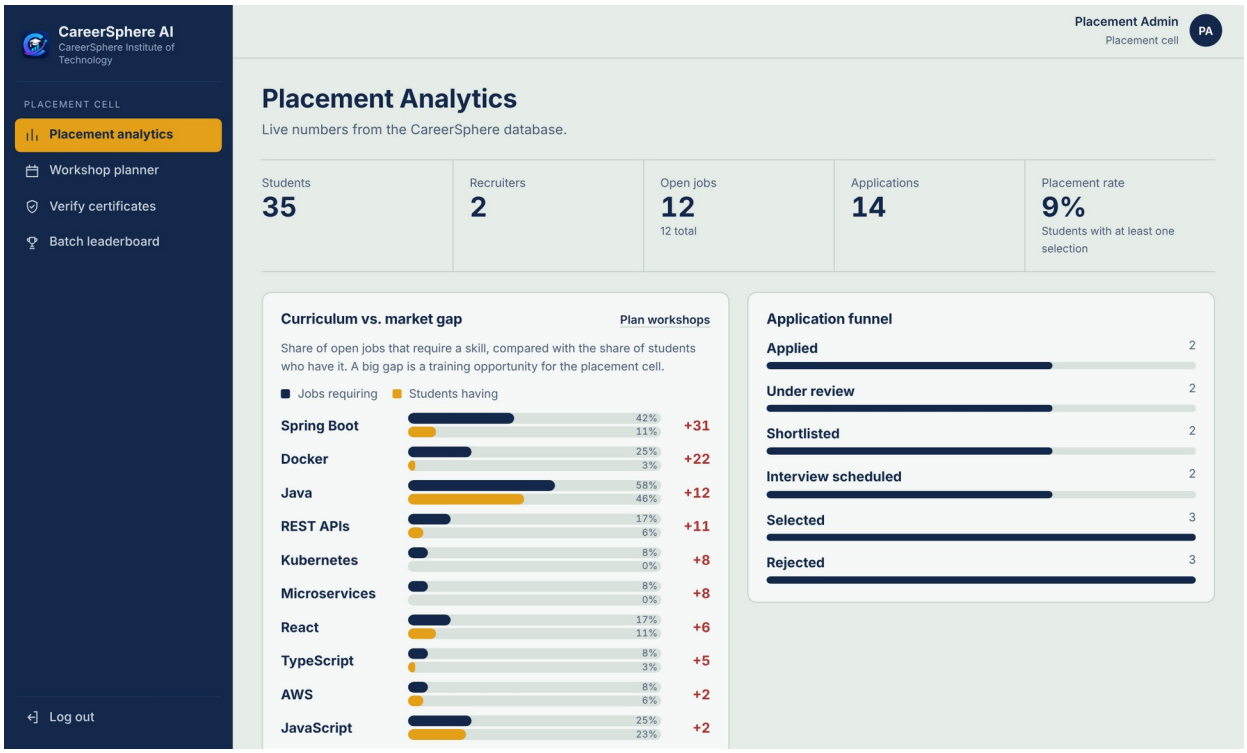


Fig 6.1.13 Placement Analytics

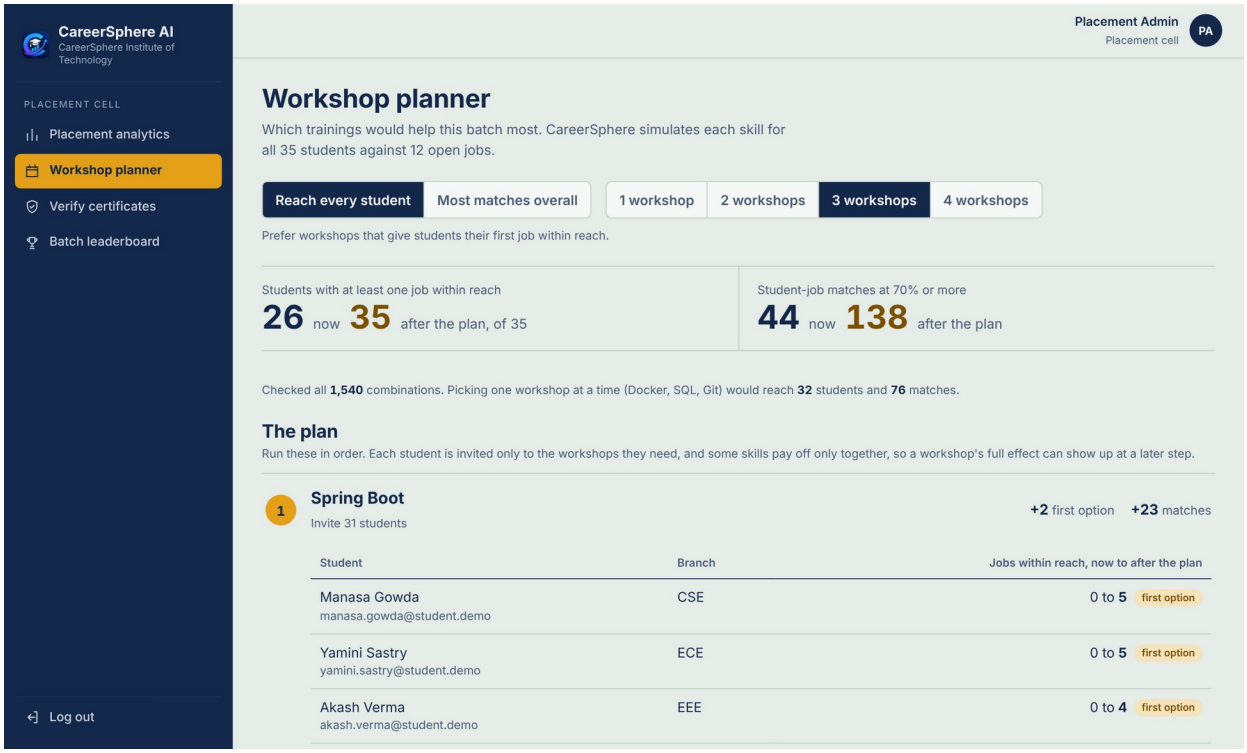


Fig 6.1.14 Workshop Planner

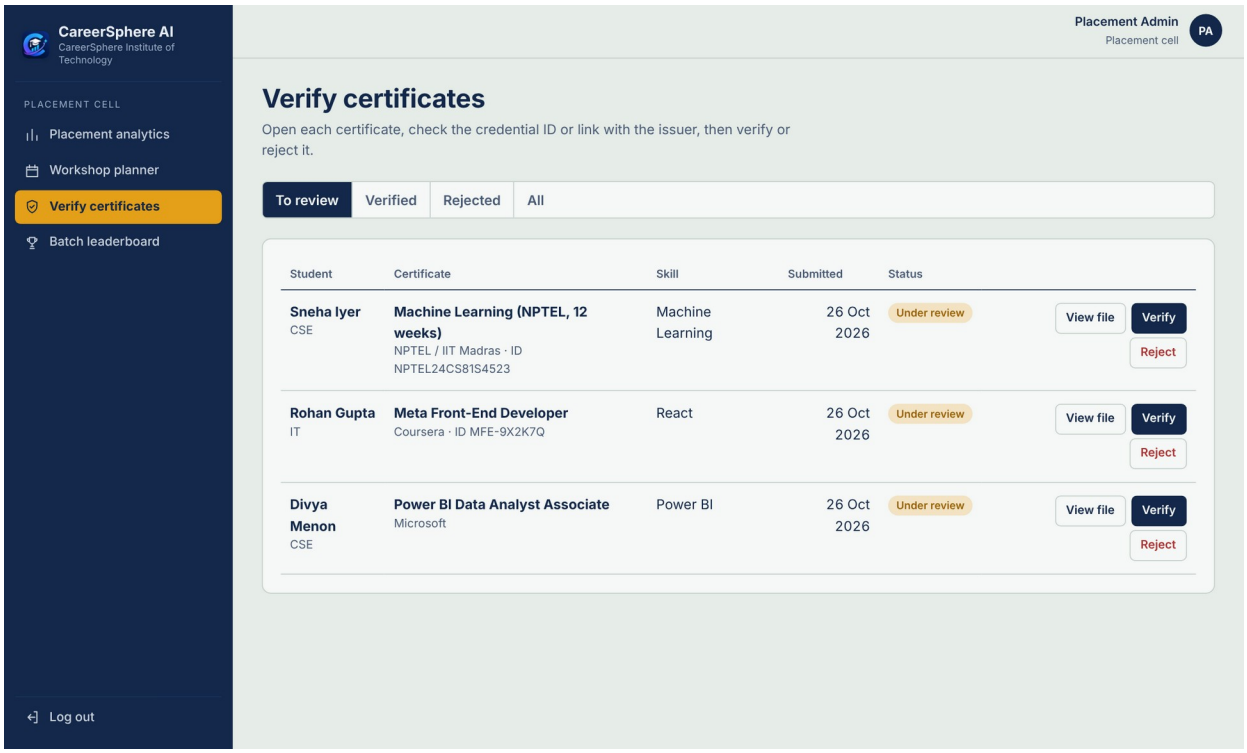


Fig 6.1.15 Certificate Verification

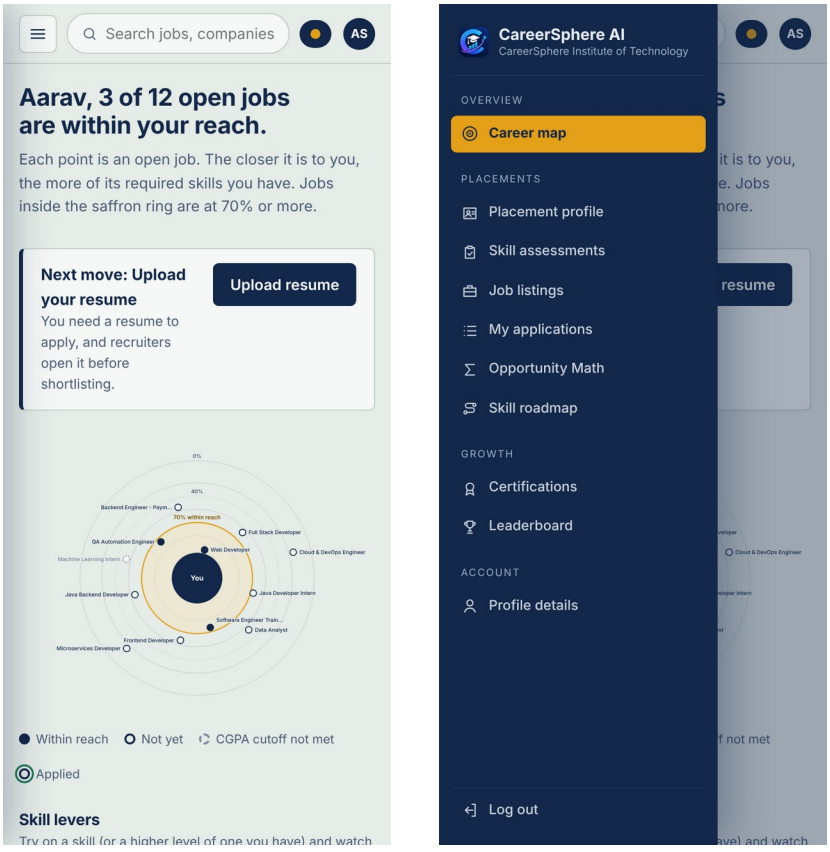


Fig 6.1.16 Mobile View

Experiment Outputs

The following figures summarise the measured behaviour of the methods on the demo batch. They are produced by running the project code; the numbers are discussed in Chapter 7.

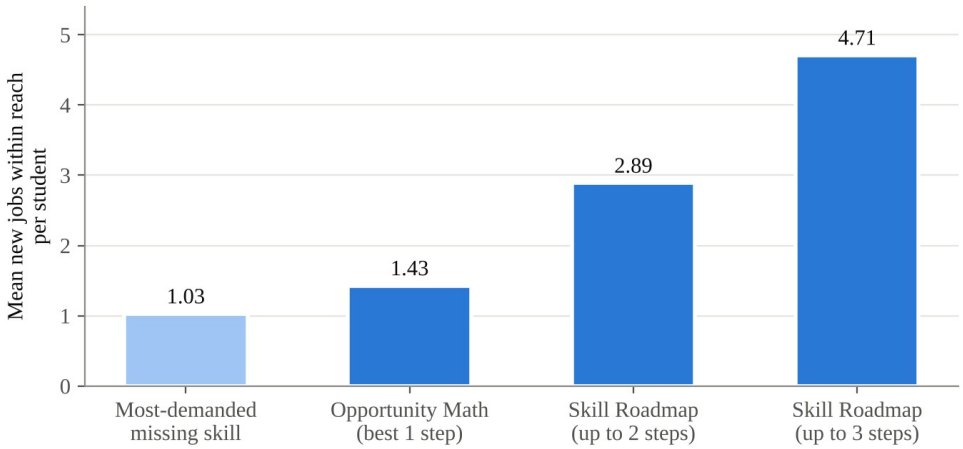


Fig 6.1.17 Learning Recommendation Comparison

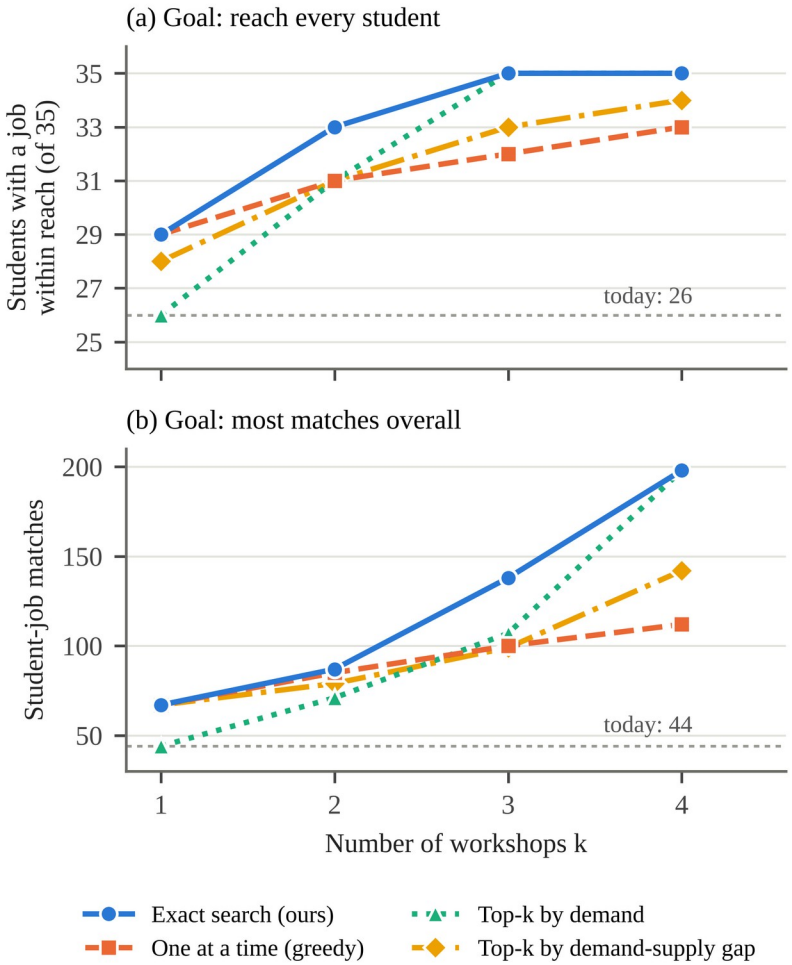


Fig 6.1.18 Workshop Planning Comparison

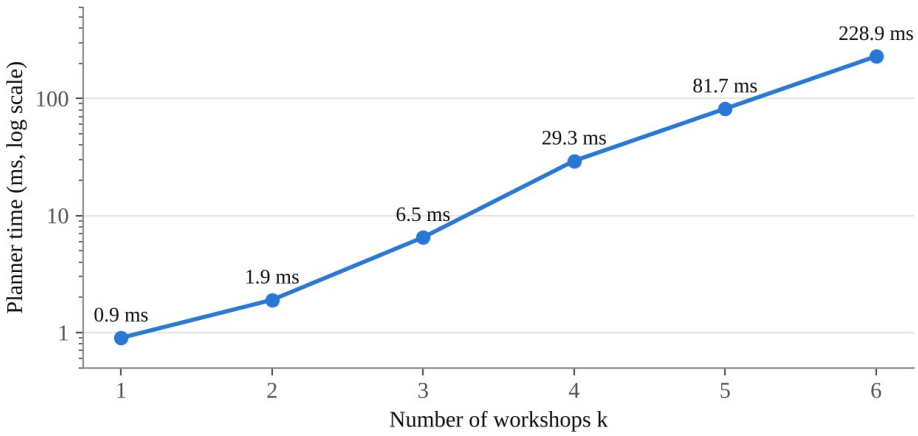


Fig 6.1.19 Runtime Comparison Graph

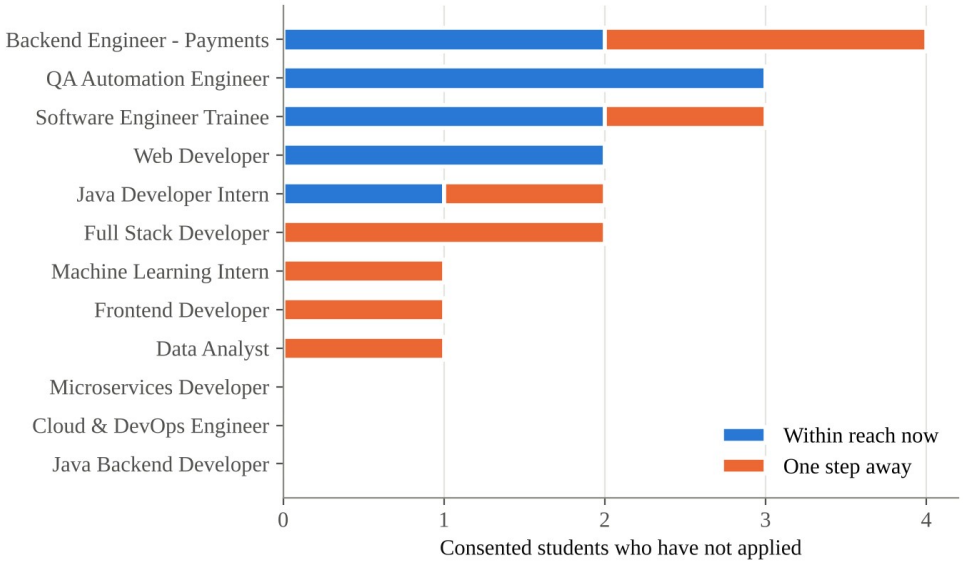


Fig 6.1.20 Hidden Talent per Job

Execution Log

The following output was produced by a verification program that loads the demo data through the real `DataSeeder` and calls the real services.

```
k=3 REACH   EXACT now 26/44 after 35/138 skills [Spring Boot, Java, Git] greedy
32/76
k=3 MATCHES EXACT now 26/44 after 35/138 skills [Spring Boot, Git, Java] greedy
29/100
Opportunity Math top pick: mean 1.43 jobs over 35 students, 7 unlock nothing
Roadmap: 35 students, mean new jobs 1-step 1.43, 2-step 2.89, 3-step 4.71;
   7 have no 1-step gain, 7 of them gain with 2 steps; 59 synergy bundles
TALENT total within 10 near 9 applications 14
39 passed, 0 failed
```

CHAPTER-7

RESULTS AND DISCUSSION

7.1 Results

The demo batch has 35 students from four branches with 3.49 skills on average and 12 open jobs from two companies requiring 4.33 skills on average, 4 of them with a CGPA cutoff. Of 420 student–job pairs, 44 are within reach today, and 26 students have at least one job within reach. Table 7.1 compares ways of choosing what a student should learn next.

Table 7.1 Learning recommendations for 35 students

Method	Mean new jobs	Students helped	Unlock nothing
Most-demanded missing skill	1.03	15	20
Opportunity Math (best 1 step)	1.43	28	7
Skill Roadmap (up to 2 steps)	2.89	35	0
Skill Roadmap (up to 3 steps)	**4.71**	**35**	0

Opportunity Math brings 1.43 jobs per student within reach against 1.03 for the most-demanded missing skill; for 20 of 35 students that skill would not bring a single job within reach. Allowing two steps doubles the gain and three steps more than triples it. All 7 students for whom no single skill helps gain at least one job with two steps.

Table 7.2 Workshop plans: students reached / matches

k	Exact	Greedy	Top-k demand	Top-k gap
1	29 / 56	29 / 56	26 / 44	28 / 67
2	33 / 65	31 / 64	31 / 71	31 / 79
3	**35 / 138**	32 / 76	35 / 107	33 / 99
4	35 / 198	33 / 81	35 / 198	34 / 142

With three workshops exact search selects Spring Boot, Java and Git and reaches all 35 students with 138 matches; greedy selects Docker, SQL and Git and reaches 32 with 76. The planner takes 6.5 ms for k = 3 and 228.9 ms for k = 6 (median of nine calls).

7.2 Discussion

RQ1 and RQ2. The usual heuristic of recommending the most-demanded missing skill often recommends a skill that unlocks nothing. Opportunity Math fixes this for one step, and the Skill Roadmap shows that the value of further steps comes mostly from combining skills: of the 65 multi-step bundles shown, 59 are marked *works together*. One student gains nothing from any one skill but three jobs from Java and SQL together. The search checks 1,083 bundles per student on average and runs on every page load.

RQ3. Greedy selection falls behind even the simple top-k demand ranking from k = 3, because Java helps few students on its own and pays off only together with Spring Boot and Git. Exact search is never worse than any baseline on its own objective; on this batch the demand ranking coincides with the optimum at k = 4, so demand is a reasonable prior but not a substitute for search.

RQ4. Table 7.3 shows how many consented students fit each job without having applied.

Table 7.3 Hidden Talent across the 12 jobs

Measure	Value
Consented students	10
Applications made by them	14
Student–job fits within reach, not applied	10
Student–job pairs one step away, not applied	9
Example: Backend Engineer – Payments	2 within reach, 2 need Docker

A recruiter would otherwise see none of these 19 student–job matches. Students without consent never appear, so the feature widens the pool without weakening the consent rule.

7.3 Existing system vs Proposed System

The existing system here means a typical placement portal and the simple heuristics used by placement cells, not a specific commercial product.

Table 7.4 Existing system and proposed system comparison

Aspect	Existing system	CareerSphere AI
Match score	None, or opaque	Explained line by line, level-aware
Next skill	Most-demanded missing skill: 1.03 jobs	Opportunity Math: 1.43 jobs
Multi-step plan	Not offered	Skill Roadmap: 4.71 jobs in 3 steps
Workshop plan (k = 3)	Greedy: 32 students reached	Exact: all 35 students reached
Recruiter reach	Applicants only	Plus 19 hidden-talent matches
Skill claims	Self-reported	Server-graded tests, checked certificates
AI output	Unchecked text where used	Tools over own data; evidence-checked

CareerSphere improves on the existing approaches on every measure tested here, but the evidence is bounded: it comes from a fictional batch, the weights and threshold are design choices, and the planner ignores capacity and cost. A study over a real placement season and the planned human evaluation are needed before any claim about real placements.

CHAPTER-8

CONCLUSION

8.1 Project Conclusion

CareerSphere AI is a complete campus placement platform for students, recruiters and the placement cell built on one transparent, level-aware match rule. It keeps the workflow of the university placement portal and adds decisions that the portal cannot make: which skills to learn, which students to invite, and which workshops to run, for whom.

Opportunity Math brings 1.43 jobs per student within reach against 1.03 for the most-demanded missing skill. Because skills pay off together, a three-step Skill Roadmap more than triples what the best single skill achieves (4.71 jobs) and helps every student, including the 7 for whom no single skill helps. Three exactly chosen workshops reach all 35 students where greedy selection reaches 32, in 6.5 ms. Hidden Talent shows recruiters 19 suitable student–job matches among consented students who did not apply.

Language models add value only where they are grounded in the same numbers: the Copilot reads only through tools over the student’s own data, and the résumé finder keeps only skills whose evidence is in the résumé. Both work without an AI key.

The project runs on a laptop with one command, from VS Code, Xcode and Android Studio and online with Docker, and is covered by 39 unit tests and 17 API tests. The conclusions apply to the demo batch and the stated assumptions; they do not establish an effect on real placements.

Future Scope

The first priority is the human evaluation already prepared (60 student–job pairs labelled blind by three people, agreement measured with Fleiss' κ [15]) and a study over a real placement season, comparing outcomes with and without the guidance features.

The weights and the 70% threshold can be learned from past outcomes once real data are available, and the question bank can be enlarged and calibrated by item difficulty so that verified levels support certification rather than screening.

Workshop planning can include capacities and costs as an integer program, which also scales beyond exhaustive search, and the roadmap can measure effort in hours with links to specific courses for each step.

Operational extensions include email and push notifications for status changes and invitations, interview scheduling with calendars, and support for several colleges with a shared recruiter directory. These are proposed extensions requiring their own implementation and testing.

REFERENCES

[1] de Ruijt, C.; Bhulai, S. Job recommender systems: A review. arXiv preprint arXiv:2111.13576 (2021). https://arxiv.org/abs/2111.13576

[2] Zhu, C.; Zhu, H.; Xiong, H.; Ma, C.; Xie, F.; Ding, P.; Li, P. Person-job fit: Adapting the right talent for the right job with joint representation learning. ACM Transactions on Management Information Systems, 9, 3, 1-17 (2018).

[3] Alfana, S.; Jammalamadaka, S. K. R.; Prakash, V. C.; Reddy, B. T. Cognitive and academic-based probability models for predicting campus placements. IAES International Journal of Artificial Intelligence, 11, 4, 1239-1251 (2022). https://doi.org/10.11591/ijai.v11.i4.pp1239-1251

[4] Zhang, Y.; Chen, X. Explainable recommendation: A survey and new perspectives. Foundations and Trends in Information Retrieval, 14, 1, 1-101 (2020). https://doi.org/10.1561/1500000066

[5] Nemhauser, G. L.; Wolsey, L. A.; Fisher, M. L. An analysis of approximations for maximizing submodular set functions—I. Mathematical Programming, 14, 265-294 (1978).

[6] Feige, U. A threshold of ln n for approximating set cover. Journal of the ACM, 45, 4, 634-652 (1998).

[7] Krause, A.; Golovin, D. Submodular function maximization. In Tractability: Practical Approaches to Hard Problems, Cambridge University Press, 71-104 (2014).

[8] Schick, T.; Dwivedi-Yu, J.; Dessì, R.; Raileanu, R.; Lomeli, M.; Hambro, E.; Zettlemoyer, L.; Cancedda, N.; Scialom, T. Toolformer: Language models can teach themselves to use tools. Advances in Neural Information Processing Systems (NeurIPS) (2023). https://arxiv.org/abs/2302.04761

[9] Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K.; Cao, Y. ReAct: Synergizing reasoning and acting in language models. International Conference on Learning Representations (ICLR) (2023). https://arxiv.org/abs/2210.03629

[10] Ji, Z.; Lee, N.; Frieske, R.; Yu, T.; Su, D.; Xu, Y.; Ishii, E.; Bang, Y.; Madotto, A.; Fung, P. Survey of hallucination in natural language generation. ACM Computing Surveys, 55, 12, 1-38 (2023).

[11] Gao, T.; Yen, H.; Yu, J.; Chen, D. Enabling large language models to generate text with citations. Proceedings of EMNLP, 6465-6488 (2023). https://doi.org/10.18653/v1/2023.emnlp-main.398

[12] Raghavan, M.; Barocas, S.; Kleinberg, J.; Levy, K. Mitigating bias in algorithmic hiring: Evaluating claims and practices. Proceedings of the Conference on Fairness, Accountability, and Transparency (FAT*), 469-481 (2020).

[13] Jones, M.; Bradley, J.; Sakimura, N. JSON Web Token (JWT). IETF RFC 7519 (2015). https://doi.org/10.17487/RFC7519

[14] Provos, N.; Mazières, D. A future-adaptable password scheme. Proceedings of the USENIX Annual Technical Conference, FREENIX Track, 81-91 (1999).

[15] Fleiss, J. L. Measuring nominal scale agreement among many raters. Psychological Bulletin, 76, 5, 378-382 (1971).

[16] Google. Function calling with the Gemini API. Google AI for Developers (accessed 4 October 2026). https://ai.google.dev/gemini-api/docs/function-calling

[17] VMware Tanzu. Spring Boot Reference Documentation, version 3.3. https://docs.spring.io/spring-boot/

[18] Meta Open Source. React documentation. https://react.dev/

[19] Oracle. MySQL 8.0 Reference Manual. https://dev.mysql.com/doc/refman/8.0/en/

[20] OWASP Foundation. OWASP Top 10: The ten most critical web application security risks (2021). https://owasp.org/Top10/